HOW TO BE HUMAN

Technical Design Document

An isometric tower-defence game, its in-house editor, the engine they share, and the validated data both read.

BUILD	**Full Production**, Development branch, post-ESV-6 / TU-9
STATUS	Living document. It describes; it is not the source of truth.
STACK	Python 3.11 · pygame-ce · PySide6 · jsonschema · pytest
SCALE	Four layers · ~33k lines · 79 data files · 22 schemas
COMPANION	SYSTEM_DIAGRAM_BRIEF.md
STUDIO	**drunken donuts**

How To Be Human: Technical Design Document

Full Production build. Isometric tower defence in Python. The player spends *love* to unlock tiles and place musicians and defenders that protect "the hole" from waves of society's pressures.

Document status	Living. Reflects the repo as of the `Development` branch, post-`ESV-6` / `TU-9`.
Supersedes	The prototype's `AI_REFERENCE.md` (`../HowToBeHuman/ClaudePrototype/HowToBeHuman/AI_REFERENCE.md`), which documents a *different, retired* codebase.
Authority	This document *describes*. It is **not** the source of truth. See §0.3.
Companion	`SYSTEM_DIAGRAM_BRIEF.md`: the visual spec for the system diagram derived from Part 2.

Table of contents

Part 0: How to read this document

0.1 Audience

Three readers, in priority order:

1. **A new engineer or agent** who needs the whole shape of the system before touching it.
2. **A designer or producer** who needs to know what is data (changeable in the editor, no code) versus what is code.
3. **A graphic designer** producing diagrams or documentation art, who needs accurate structure without needing to read Python. That reader should start at Part 2 and then read `SYSTEM_DIAGRAM_BRIEF.md`.

0.2 Structure

Parts 3–6 mirror the repository's four top-level packages, in dependency order: **engine → data → game → editor**. Each subsystem section follows the same five-heading shape so you can skim laterally:

> **Owns**: the one-sentence responsibility. **Key types**: the classes/functions you will actually touch. **Contract**: what callers may rely on. **Invariants**: what will break if you change it carelessly. **Source**: files and the owning `CLAUDE.md`.

0.3 What is authoritative

This document is a **map, not the territory.** When it disagrees with the repo, the repo wins. The authority chain is:

RANK	SOURCE	OWNS
1	`data/**.json` + `data/schemas/**`	Every gameplay value, every asset binding, every map, every UI layout override.
2	`SPEC.md`	Numbered requirements (`E-*` engine, `D-*` data, `G-*` game, `ED-*` editor, `T-*` tooling).
3	The package `CLAUDE.md` routers and subsystem docs	Architecture decisions and their reasoning.
4	**This document**	Synthesis and orientation.
-	`../HowToBeHuman/ClaudePrototype`	**History, not authority.** Readable for archaeology ("why is this number 240?"). Never edited from here. Nothing in the test suite compares against it.

0.4 Conventions used here

- `monospace` = a real identifier, path, or JSON key you can grep for.
- **DATA** in a margin note = this is a designer-editable value, not code.
- ⚠️ = a load-bearing invariant; breaking it produces a subtle, non-crashing bug.
- Requirement tags like **(E-21)** cross-reference `SPEC.md`.
- "×10 combat scale" = HP and damage values in `data/` are stored ten times their conceptual value, so designers get integer granularity. `core.json TheHole.base_hp` is the one deliberate exception and stays at absolute `10`.

Part 1: The product

1.1 The game in one page

Genre. Isometric, grid-based, wave-defence with a build phase.

Premise. You keep a small colony of musicians alive. Enemies (society's pressures) march in from the spawn band toward *the hole*, the colony's heart. You cannot stop them by walling up; you slow them, out-damage them, and out-earn them.

The currency is love. You spend it to unlock tiles, place buildings, and upgrade them. It arrives once per round, at payday.

The loop.

```
BUILDING          you place / upgrade / unlock.       (interactive, untimed)
   │ End Turn
   ▼
ENEMY             the wave spawns and walks.          (simulated, speed 1×/1.5×/2×/pause)
   │ wave cleared, or the hole is breached
   ▼
ROUND_END         a short beat.                       (timed)
   │
   ├─► BOSS_CUTSCENE   every Nth round: an A/B story choice.
   ├─► LEVELUP         if XP crossed the village threshold: pick 1 of 3 rewards.
   ▼
INCOME            payday: love lands, buildings revive.
   │
   ▼
BUILDING          round + 1
```

Losing. The hole has *lives*, not hit points. Each breach costs one life and wipes the rest of the wave. At zero lives the run ends. (The prototype's HP-based hole is retired.)

Winning. Not implemented. Survival run.

Progression has two independent ladders:

LADDER	CURRENCY	GRANULARITY	EFFECT
Per-building upgrade	love	one building at a time	+HP, +damage, +yield within its current tier
Village research	XP → level-up choice	global per building *type*	unlocks a type, or unlocks its next tier so every building of that type can be upgraded further

A building can never advance into a tier that has not been *researched*. This is the whole mid-game economy: love buys width, XP buys depth.

1.2 Deliverables & stack

The game	Windows one-folder PyInstaller build → `dist/HowToBeHuman/HowToBeHuman.exe`. Run from source: `py game/main.py`.
The editor	Run from source only: `py editor/main.py`. The designer's single interface to every piece of game data.

Language	Python 3.11+
Game rendering	`pygame-ce`
Editor shell	`PySide6` (Qt 6)
Image slicing	`Pillow`
Validation	`jsonschema` (draft 2020-12)
Video	`opencv-python` : **optional**; absent ⇒ cutscenes silently skip
Tests	`pytest` + `pytest-xdist` over `unittest.TestCase` classes

1.3 Design pillars

These are the tie-breakers for every architectural decision.

1. Agent legibility. Small single-purpose files. Schemas instead of naming conventions. No state that only the editor can see. The codebase is optimised for an AI agent (or a new human) reading one file and being correct about it.

2. Strict layering. Game logic never touches pygame. `editor/` and `game/` never import each other. Both consume `engine/` and `data/`. This is what makes the game headlessly testable and the editor's viewport honest.

3. The editor is the designer interface. Humans never hand-edit `data/` JSON. Agents may, but only schema-valid writes through the validating writer. Every value a designer can change is reachable from a panel.

1.4 Non-goals

- **Not a general-purpose engine.** `engine/` carries exactly this game's workload and nothing more.
- No forces/impulse physics. No collision response.
- No procedural sprite generation at runtime. The prototype's `sprite_gen` is not ported; the grey-X placeholder is the only "no asset" render.
- No in-process play-in-editor. **Play is always a detached subprocess.**
- No multiplayer. No non-Windows packaging (for now).
- No autotiling in the map editor. The checkerboard `_b` variant is a position-parity rule, not terrain matching.

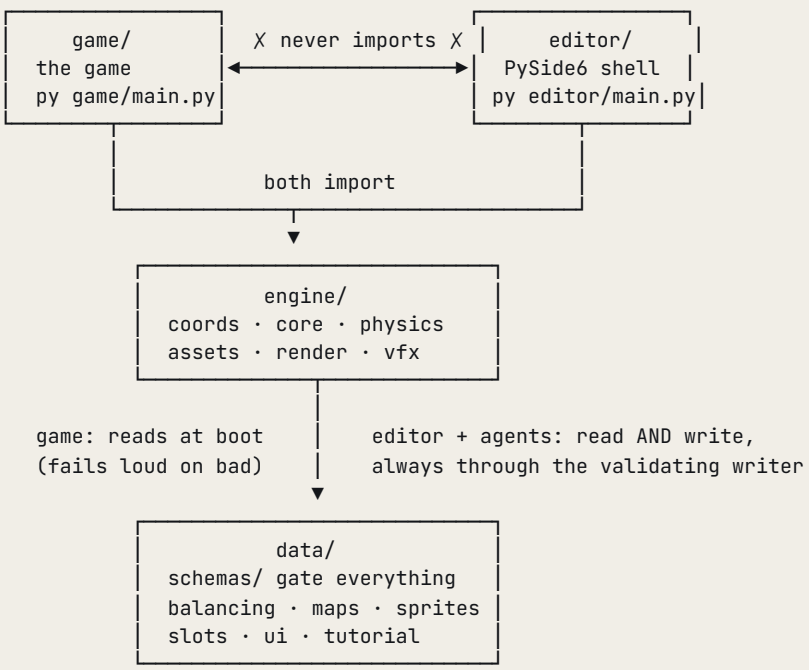
1.5 Glossary

TERM	MEANING
Love	The single currency. Earned at payday, spent on unlocks / builds / upgrades.
The hole	The player's base. A single map object, movable in the editor. Lives-based, not HP-based.
World space	Fractional tile coordinates `(col, row)` . **The only space game logic uses.**
Screen space	Pixels after isometric projection + camera pan + zoom.
Slot	A named asset attachment point, e.g. `stone_thrower_t1_lvl1` , `tile_buildable` , `vfx_explosion` . Declared in `data/slots.json` .
Manifest	`data/sprites/asset_manifest.json` : maps slots to sheets + animation rows. "v2".

TERM	MEANING
Domain	One of `buildings / enemies / map / ui / core / vfx`: the unit of balancing files and file ownership.
Category	A slot-registry grouping. The six domains plus asset-only `deco`, `conditions`, `backgrounds`.
Active map	The one map the game loads, pointed at by `data/maps/active_map.json`.
Zone	A tile's gameplay state: `BUILDABLE` / `BUILT` / `COMBAT` / `SPAWNING` / `BACKGROUND`.
Condition	A tile's terrain flavour, rolled once at map load: `GRASS` / `MOUNTAIN` / `POND` / `FOREST`. Modifies stats and path weight.
Tier / level	A building has 3 tiers; within a tier it has levels. Tiers are researched globally; levels are bought per building.
Era	A boss's scaling index (`round // interval - 1`), and by extension the era-indexed sprite/stat tables.
Footprint	How many tiles wide an enemy's N×N block is. Anchored at the block's **min corner**.
Payday	The `INCOME` phase's twelve-step ordered settlement.
Slot 6/7/8/…	A payday step, by its ordinal position. The ordering is sacrosanct (§5.3).
Placeholder	The grey-X sprite drawn for any slot with no imported art.
Handoff	A schema-validated JSON brief the editor writes to dispatch an agent task.

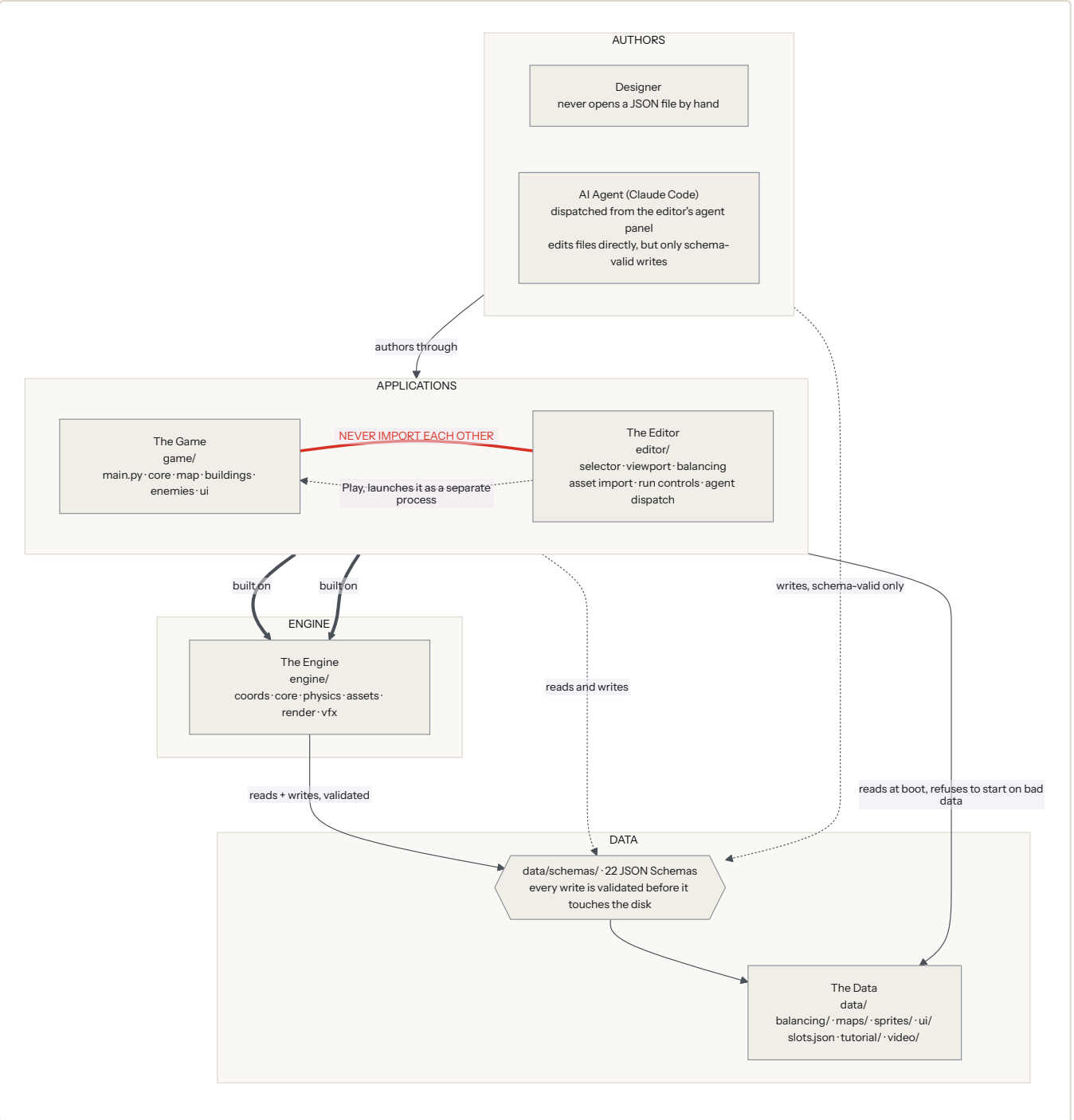
Part 2: System architecture

2.1 The four layers



Why this shape. The bottom layer is the only value store. The engine is game-agnostic (it contains no word like "raider" or "flute"). The two applications are peers that never see each other, which is precisely what lets the editor's viewport be *honest*: it draws through the same `engine/render` pipeline the game does, so what a designer sees is what ships.

The system map. The same four layers as a wiring diagram. Read it top to bottom: authors act on the applications, the applications stand on one shared engine, and every write lands on validated data.



How to read the arrows.

LINE	MEANS
thick solid	is built on. The dependency spine.
solid	reads / uses
dotted	writes. An authoring action.
dotted, labelled `Play`	launches as a separate OS process, not as part of itself
red, no arrowheads	must never connect. This is the *absence* of a link, not a link.

Arrows point from the thing that needs something to the thing it needs, so every arrow runs downward or sideways. `Play` is the one exception and is deliberately drawn as a different *kind* of relationship: a process launch, never an import.

The five facts the map exists to make obvious:

1. **Data is the foundation, and everything is validated.** Every file has a schema. Nothing reaches `data/` without passing it.
2. **The engine is shared.** Both applications sit on the same one, which is why the editor can show a truthful preview of what the game will draw.
3. **The game and the editor never import each other.** A hard architectural rule, and the single most distinctive thing about this codebase.
4. **The editor still launches the game**, but as a completely separate process. That is a different kind of relationship from "imports".
5. **Two kinds of author exist:** human designers, who work through the editor, and AI agents, who edit files directly but only through the same gate.

2.2 Dependency rules

These are enforceable by review and by tests (`test_editor_viewport.TestPurity`, the `game/ui` source-scan purity test, `test_core.py`'s import checks).

RULE	ENFORCEMENT
`game/` never imports `editor/`; `editor/` never imports `game/`.	`TestPurity` import list: **every new editor module must be added to it**.
`engine/coords`, `engine/core`, `engine/physics`, `engine/tilemap.py`, `engine/data_io.py`, `engine/vfx`, asset *metadata* code import **no pygame**.	The allow-list in §3.8; purity tests.
All iso math lives in `engine/coords`. Nothing else may compute a projection.	Review. Both hosts route picking/panning/zooming through `screen_to_world` / `world_to_screen`.
All disk writes to `data/` go through `engine.data_io.write_validated`.	The editor has exactly one write path per file type; agents are instructed the same.
`game/ui` may import `game/core`: **one-way**. Never the reverse.	`game/ui/shell.py` lives in `ui`, not `core`, specifically to avoid the cycle.
`game/core` imports nothing from `game/enemies`.	Cross-boundary needs are **optional callbacks** (`on_base_hit`, `on_enemy_death`, `on_kidnap`, `on_splash_impact`, `on_defender_fire`, `on_projectile_hit`) and **opaque payloads** the receiver never inspects.
`game/map` imports nothing from `game/buildings`.	It **duck-types** occupants (`alive`, `building_type`, `wall_hp()`, …).
The engine contains no game vocabulary.	Review. `fit_tiles` / `scale` in the renderer are the engine-generic names for the game's `footprint` / `sprite_scale`.

Two sanctioned duplications, both because `editor/` may not import `game/`:

- `editor/panels/_screen_primitives.py` re-implements `game/ui`'s unskinned widget look for the screen-mode preview. Kept aligned by eye + a parity pin.
- `editor/vfx_params.py` mirrors `game/ui/effects.py`'s `_params_from_balance` JSON-key→dataclass adapter. Do **not** resolve this by moving the mapping into `engine/vfx`: that would give the engine JSON vocabulary.

2.3 The two hosts

Everything that is *not* pure logic lives in a host.

CONCERN	GAME HOST (`GAME/MAIN.PY`)	EDITOR HOST (`EDITOR/MAIN.PY`)
Window	real SDL window, size from `data/display.json`	Qt `MainWindow`; the render target is an **offscreen** `pygame.Surface` converted to `QImage`
SDL driver	real	`SDL_VIDEODRIVER=dummy` set at module level in `panels/viewport.py`, before `import pygame`
Input mapping	keyboard + mouse → camera + click ladder	Qt events → tools, brushes, undo stack
Frame driver	`pygame.time.Clock` at `display.json` `fps`	`QTimer` at 16 ms
Owns	the `_World` (scene, session, tilemap, screens)	the selection model, the undo stacks, the sessions
Data	reads at boot, **fails loud** on invalid	reads and writes, always validated

> ⚠️ The editor's `SDL_*=dummy` mutation is a real `os.environ` write that **every subprocess inherits**. `run_controls._real_window_environment()` strips it before launching Play/Playbuild/`claude`: without that strip, Play runs correctly but renders into an invisible surface.

2.4 Three journeys through the system

Journey A: Boot (the game starts)

```
py game/main.py
 ├─ load data/display.json                    → window size, fps, caption
 ├─ load data/geometry.json                   → tile pitch (64×32)
 ├─ load data/balancing/core.json Camera      → zoom levels [1,2,4], default
 ├─ load data/maps/active_map.json → maps/<id>.json
 │    engine.tilemap.load_map: schema-validate, then cross-check
 │    (row counts vs dims, markers in bounds, id == filename stem)
 ├─ build CoordinateSystem with THIS map's cols/rows
 ├─ load data/slots.json                      → SlotRegistry   (fails loud)
 ├─ load data/sprites/asset_manifest.json     → Manifest       (degrades)
 ├─ build AssetStore(manifest, registry, …)
 ├─ load data/ui/fonts.json + palette.json + strings.json + active_font.json
 │    → configure_fonts / configure_palette / configure_strings   (fail loud)
 ├─ load data/balancing/{buildings,enemies,map,ui,core,vfx}.json  (fail loud)
 ├─ load data/ui/screens/*.json               → ScreenSkinning
 ├─ load data/tutorial/tutorial.json          → TutorialDirector (degrades)
 ├─ load data/video/cutscenes.json            → CutscenePlayer  (degrades)
 └─ Shell → CUTSCENE → MAIN_MENU → (New Game) → build_gameplay()
```

The **fail loud vs degrade** split is deliberate and is a rule, not a habit, see §7.4.

Journey B: One frame of gameplay

```
dt = clock.tick(fps) / 1000
sim_dt = dt * session.combat_speed   if phase == ENEMY else dt      ← ONE value
                                                                       feeds all
                                                                       three sim calls
1  INPUT        pygame events → click ladder (§5.13) / camera / keys
2  SIM          session.pre_sim(sim_dt, scene)     spawner, phase timers, payday
                scene.update(sim_dt)               spawn queue → components → despawn queue
                resolve_combat(scene, tilemap, sim_dt, …)   targeting, firing, deaths
                session.post_sim(scene)            wave-clear, death-spawn flush
3  SUBMIT       ground_cache.ensure(...)           scrolling composited ground
                condition_render_items(...)        terrain layer
                scene.render_items()               entities
                spawn_deco / map deco              deco layer
                overlay pass                       ranges, highlights, tints, grid
                HUD pass                           screens, floaters, bars
4  FLUSH        renderer.flush(window)             depth sort → batched blits
5  FLIP
```

⚠️ `sim_dt` is computed once and passed to all three sim calls. Splitting it would desync the spawner, movement, and combat at 1.5×/2×.

⚠️ `session.frozen` (LEVELUP / BOSS_CUTSCENE) skips step 2 entirely: no updates, no animations behind a modal.

Journey C: A designer changes something

```
Designer opens the editor
 └─ selects ONE node in the tree                ← the whole editor hangs off this
      ├─ a map leaf       → tilemap mode  → paint → QUndoStack → Save → maps/<id>.json
      ├─ a UI screen leaf → screen mode   → drag widgets → ui/screens/<id>.json
      └─ an entity leaf   → entity preview
           ├─ Balancing panel  → staged edits + dirty dots → Save → balancing/<domain>.json
           │                                                  + balancing_history/<domain>.json
           └─ Details panel    → Import Spritesheet → rows/fps/loop/offset/anchors
                                                    → sprites/imported/<slot>.png
                                                    → asset_manifest.json
 └─ Play ▶  saves + validates, then launches `py game/main.py` DETACHED
```

Every write in that diagram goes through `engine.data_io.write_validated`, which schema-validates before touching the disk and dumps deterministically (sorted keys, 2-space indent, trailing newline) so git diffs stay minimal.

2.5 Scale of the system

Approximate, as a sense of proportion for anyone sizing a diagram or a task.

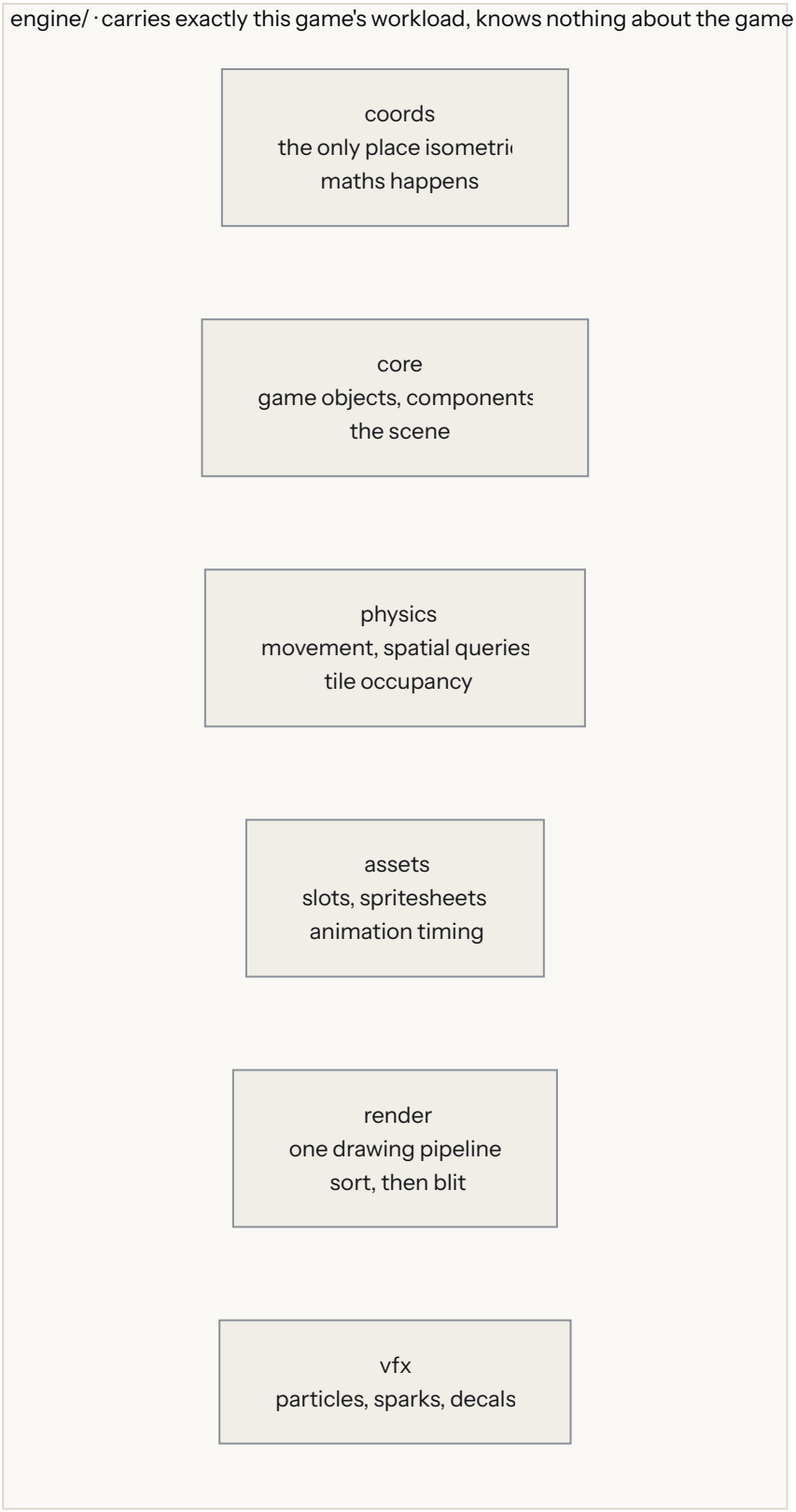
PACKAGE	PYTHON FILES	LINES	CHARACTER
`engine/`	44	~4,400	small, dense, heavily invariant-laden
`game/`	69	~14,600	the bulk of the behaviour
`editor/`	46	~13,400	Qt panels, roughly game-sized
`tools/` (incl. tests)	115	~32,700	103 test modules
`data/`	79 JSON files	-	22 schemas, 6 balancing domains, 8 maps, 14 UI screens

---

Part 3: The engine (`engine/`)

A *pseudo*-engine: it carries exactly this game's workload. Requirements: `SPEC.md` §4 (`E-*`). Router: `engine/CLAUDE.md`.

The six subsystems.



Everything above is pure Python except the parts named in the pygame allow-list (§3.8). That split is what keeps game logic headlessly testable.

3.1 `engine/coords`: the coordinate authority

> **Owns**: every isometric projection in the entire repository. **(E-1…E-5)**

Key types. `CoordinateSystem`, `Camera`, `load_coordinate_system(data_dir, …)`.

Contract.

CALL	RESULT
`world_to_screen(wx, wy)`	`(px, py)` for *fractional* world coords; applies iso projection, pan, zoom
`screen_to_world(px, py)`	exact inverse; round-trip error < 1e-6 at zoom 1 **(E-3)**
`depth_key(...)`	the iso draw-order key, used only by the renderer
`clamp(...)`	keeps the viewport on the map; *centres* an axis when the map is smaller than the viewport there
`center_on(wx, wy, w, h)`	parks a world point at the viewport centre, then clamps
`visible_tile_window(vw, vh, margin)`	integer `(col_min, col_max, row_min, row_max)` of tiles that can touch the viewport
`set_zoom(z)`	must be a member of the configured `zoom_levels`, else `ValueError`

The projection.

```
screen = iso * zoom - pan
ix = (wx - wy) * half_w        half_w = tile_w / 2 = 32
iy = (wx + wy) * half_h        half_h = tile_h / 2 = 16
```

World `(0,0)` is the **top corner** of tile `(0,0)`'s diamond. Camera pan is in screen pixels.

Geometry sources: **DATA**

VALUE	FILE	SHIPPED
`tile_w`, `tile_h`	`data/geometry.json`	64 × 32
fallback `map_cols` / `map_rows`	`data/geometry.json`	20 × 20
`zoom_levels`, `default_zoom`	`data/balancing/core.json` `Camera`	`[1.0, 2.0, 4.0]`
per-map dims	each `data/maps/<id>.json`	4…1024 per axis

Zoom is a *balancing tunable*, not a geometry constant: real hosts always pass it as an override, exactly the way each map passes its own dims. The loader cross-checks the relationship (`default_zoom ∈ zoom_levels`) that the schema cannot express.

Invariants. ⚠️ `visible_tile_window` is what lets a 1024×1024 map render at full fps; both hosts must window their tile submission through it. ⚠️ Pure Python, no pygame: this is what keeps picking and camera logic headlessly testable.

Source. `engine/coords/{system,camera,geometry}.py` · `engine/coords/CLAUDE.md`

3.2 `engine/core`: the game-object model

> **Owns**: `GameObject`, `Component`, `Transform`, `Scene`, frame ordering. **(E-10…E-15, E-30/31 wrappers)**

The rule, stated once: *components are what the editor sees; subclasses are behaviour convenience.* **All gameplay state lives in declared component fields.** Never add authoritative state as a plain subclass attribute.

Key types.

TYPE	ROLE
`GameObject`	stable id, name, tags, `Transform` (world pos + layer), ordered component list, `on_spawn` / `on_update(dt)` / `on_despawn`
`Component`	class-level annotated fields with defaults, collected by `__init_subclass__` into `cls._fields`; JSON-safe types only (bool/int/float/str/list/dict); auto-registered by name
`Transform`	`wx`, `wy`, `layer`
`Scene`	owns objects + a `SpatialGrid`; spawn/despawn queues applied at frame boundaries; `by_type` / `by_tag` / `queued_by_tag` / `query_area` / `query_chebyshev` / `render_items`

Shipped components. `SpriteAnimator` (slot key, animation, phase offset, `fit_tiles`, `scale`), `Health`, `Movement` (waypoint follower), `RangeSensor` (Chebyshev radius).

Frame boundaries (E-13). `Scene.update(dt)`: 1. apply the spawn queue (`on_spawn` fires) 2. rebuild the spatial grid 3. update live objects **in spawn order**; within an object, **components in list order**, then the subclass `on_update` hook 4. apply the despawn queue

> ⚠️ **`queued_by_tag(tag)` exists because of a real bug.** `spawn()` only *queues*; `by_tag()` reads the *live* list. Any caller that spawns and then asks "is anything of this kind left?" **within the same frame** cannot see what it just spawned. This silently ended a round under the children of an enemy that burst on the wave's last frame. The wave-clear check now consults both.

> ⚠️ **The setattr guard (E-11).** After `GameObject.__init__`, assigning a *new public* attribute raises `AttributeError`. Underscore-prefixed transients are allowed (never serialised, non-authoritative). Consequence you will hit: **a public property cannot have a setter**: that is why the codebase uses methods like `mark_death_spawned()` instead of `death_spawned = True`.

Serialisation (E-15). `to_dict()` → `{id, name, tags, transform:{wx,wy,layer}, components:[{type, fields}]}`. `from_dict` returns a *base* `GameObject`: subclass identity is not persisted, because components carry all the state. `game/buildings/registry.create()` reconstructs the subclass.

The owner seam. `Component.on_added(owner)` is a no-op by default; a component needing its owner's transform caches `self._owner = owner` there.

Render submit hook. A component with a visual presence defines `render_items(transform)` → `iterable[RenderItem]`. `Scene.render_items()` collects generically. `engine.core` may import `engine.render.item` (pure data): still no pygame.

Source. `engine/core/*.py` · `engine/core/CLAUDE.md`

3.3 `engine/physics`: three primitives, deliberately

> **Owns**: waypoint movement, spatial queries, tile occupancy. **(E-30…E-32) Do not grow forces or collision response without the user asking.**

PRIMITIVE	CONTRACT
`SpatialGrid(cell_size=1.0)`	buckets by `(floor(wx/cell), floor(wy/cell))`. `query_radius` (Euclidean, candidate cells then exact test) and `query_chebyshev` (tile = `round(wx), round(wy)`). Returns in **insertion order**: deterministic. Cell membership is fixed at insert/rebuild; exact tests read *live* positions.
`TileOccupancy`	`(col,row) → obj`, one occupant per tile. `set/clear/get/is_occupied`.
`advance(pos, waypoints, index, speed, dt, threshold=0.06)`	pure waypoint step → `(new_pos, new_index, arrived_this_step, reached_end)`.

> ⚠️ `advance` **clamps the step to the remaining distance**, a deliberate divergence from the prototype. Without the clamp a step larger than `threshold` overshoots and the next call walks *back* onto the waypoint (a visible per-tile reversal); at `2*threshold` the unit locks into permanent oscillation with `index` never advancing.

> ⚠️ **At most one waypoint is consumed per call.** `PathAgent` checks the next tile for a blocker/wall once per frame *before* `Movement` runs; skipping several waypoints in one call would let a unit tunnel past a check that never happened.

Source. `engine/physics/*.py` · `engine/physics/CLAUDE.md`

3.4 `engine/assets`: slots, manifest, placeholder

> **Owns**: the data-driven slot registry, manifest v2 parsing, frame resolution, the grey-X placeholder. **(E-33…E-38)**

Two-level model.

```
data/slots.json          "which slots exist"     → SlotRegistry   (fails LOUD)
  categories[] → groups (recursive tree) → slots[]
  each category: key, display_name, frame_w, frame_h, animations[]

data/sprites/asset_manifest.json   "what art is bound to them"  → Manifest (degrades)
  entries[slot] = { sheet, frame_w, frame_h, offset_x, offset_y,
                    rows[], slice?, anchors?, tint_overlay? }
  rows[i] = { animation, frames, fps, hidden[], loop_start, loop_end, loop_count }
```

Row semantics: prototype-exact and non-negotiable. Rows are animations. **Row 0 is `idle` and is required** (schema-enforced via `prefixItems`). `fps → max(1, round(1000/fps))` ms per frame. `playback_order` = pre-roll → looped range × `loop_count` → post-roll, with `hidden` frames dropped **after** expansion.

Frame resolution.

CALL	BEHAVIOUR
`Manifest.current_frame(slot, animation, time_ms, phase_ms=0)`	pure function of time → `(row, col)` or the `PLACEHOLDER` sentinel. Missing animation **falls back to idle**. Missing slot / no usable idle → `PLACEHOLDER`.
`Manifest.animation_ms(slot, name)` / `AssetStore.animation_total_ms`	a named track's `total_ms`, or `None` when absent. **No idle fallback**: the caller uses absence as a signal (no `death` row ⇒ no corpse; no imported art ⇒ run the procedural VFX fallback).
`AssetStore.frame(slot, animation, anim_time_ms)`	the concrete `Frame`. Sheets load via `pygame.image.load` with **no** `convert()` (they need a display; the editor runs SDL dummy). Sliced frames are **subsurfaces**: the parent sheet must stay cached.

CALL	BEHAVIOUR
`AssetStore.anchor(slot, name)` / `.offset(slot)`	manifest metadata; degrade to `None` / `(0,0)`, never raise.
`AssetStore.hit_opaque(...)`	pixel hit-mask for skinned buttons (§3.5, A8).

> ⚠️ **There is no cache invalidation.** When the manifest changes, build a new `AssetStore`. The editor's `reload_assets()` does exactly that.

Frame-size precedence. manifest entry > registry per-slot override > registry category > `frame_sizes` > default.

Per-slot frame-size override. A `slots[]` entry is either a bare key string (inherits the category size) or `{key, frame_w, frame_h}`. It describes **slicing, not drawing**: on-screen size comes from the renderer's fit (§3.5). The object form is normalised away at parse time; `GroupNode.slots` stays a tuple of key strings downstream. ⚠️ A key repeated across groups of one category must **agree** on its frame size, or the loader raises: `uniqueItems` compares whole values, so `"foo"` and `{"key":"foo",…}` are two distinct items to the schema.

Three optional per-entry keys.

KEY	SHAPE	CONSUMER
`slice`	`[left, top, right, bottom]` frame-px nine-slice margins	**HUD sprites only**; world sprites keep uniform zoom scaling
`anchors`	`{muzzle?, impact?, hp_bar?, floater_origin?, status_icon?, beam_endpoint?}`, each `[x,y]` frame-px	pure metadata; never touches the blit path
`tint_overlay`	bool	tile-condition art: "keep drawing your own flat colour overlay under this sprite"

Anchors are measured on the sheet frame at frame resolution, never at draw resolution or a zoom. `+x` right, `-y` up, relative to the sprite's drawn centre. `muzzle`, `impact` and `hp_bar` have live read-sites; `floater_origin` / `status_icon` / `beam_endpoint` are declared and parse-ready but inert, shipped early so their future wiring needs no schema migration.

Tolerance (E-37). `load_manifest` **never raises**: absent file → empty manifest (the normal pre-import state); corrupt file → warn + empty; corrupt entry → warn + skip that entry. `load_registry` **fails loud**: the registry is infrastructure, like `geometry.json`. `tools/smoke.py` still fails loud on an invalid *committed* manifest; that is a separate concern.

Source. `engine/assets/{types,manifest,registry,store,placeholder,nine_slice}.py` · `engine/assets/CLAUDE.md`

3.5 `engine/render`: submit → sort → blit

> **Owns**: the one render path. **Target-agnostic: the game window and the editor viewport use it identically. (E-20… E-26)**

Flow.

```
RenderItem(slot_key, animation, anim_time_ms, world_pos, layer,
           tint?, flip?, fit_tiles, scale)
     │  submit()
     ▼
Renderer  ── resolve frame via AssetStore ──► DrawCall
     │      depth-sort
     │      convert world → screen via CoordinateSystem
     ▼
backend.py (pygame)  ── batched target.blits(...) ──► Surface
```

`renderer.py` is **pure orchestration**; the pygame backend is lazily imported on first `flush()` and is injectable for tests.

Draw layers (E-26), fixed order.

#	LAYER	CONTENT
0	`ground`	map terrain tiles
1	`terrain`	tile-condition art (mountains, ponds, forests): over the ground, under everything an entity draws
2	`entities`	buildings, enemies, the base: iso depth-sorted
3	`deco`	placed deco props + spawn-band trees: **always above entities** (enemies walk partly behind the treeline)
4	`overlay`	range circles, tile highlights, tints, grid, splatters
-	HUD	drawn by the host after `flush`, in screen space, no depth sort

> ⚠️ **`depth_key = (layer_index, wx+wy, wy).`** The draw *layer* is the primary sort key, so the whole `ground` layer always draws before everything else. **This is load-bearing**: if `depth_key` ever interleaves layers, the ground cache breaks.

The anchor convention (ER-1). A frame blits **centred on the tile**: horizontally on its world position, vertically on the tile diamond's centre.

```
dest_y = world_to_screen(wx, wy).y + (tile_h/2)*zoom - frame_h/2
```

This is *continuous in* `frame_h`. It is not a new rule: it is the old two-branch rule (32px-tall frames anchored one way, 96px another, with a 32px cliff in between) written out and simplified. Every non-enemy world frame that ships is 96 or 32 tall, so it is **byte-identical** for buildings/tiles/deco/core; the enemy sheets (18/26/28/56/84/88 tall) are the intended moves.

Sizing (ER-1), downscale-only.

```
fit = min(1.0, (fit_tiles * tile_w) / frame_w)   if fit_tiles > 0 else 1.0
s   = fit * scale            # w, h and manifest offsets all ride s
```

`fit_tiles` = how many tiles wide the thing may span; the sprite scales **down** to fit and **never up**. `scale` multiplies *after* the fit: the deliberate knob for low-res art, and it may exceed 1. ⚠️ Hard invariant: `fit_tiles == 0.0 and scale == 1.0` ⇒ `s == 1.0` ⇒ pixel-identical to pre-ER-1. Buildings, tiles, deco and the HUD pass are provably untouched.

Multi-tile block centring (ER-5). A `fit_tiles`-wide unit is *addressed* by its anchor (the block's min corner, the tile it paths to) but must be *drawn* on the block's centre: `block_center_offset(fit_tiles) = (fit_tiles - 1) / 2` tiles on both axes. Added to both axes it cancels in the iso x term and lowers y by `(fit_tiles - 1) · tile_h/2` (0px at 1, 16px at 2, 32px at 3). ⚠️ It shifts the **blit only, never** `depth_key`: folding it into the sort would move draw order with position.

Three exported helpers exist so no consumer ever restates the math: `fit_factor(frame_w, tile_w, fit_tiles)`, `block_center_offset(fit_tiles)`, and `sprite_anchor_screen(cs, wx, wy, frame_w, fit_tiles, scale, offset_xy, anchor_xy)`: *the* one shared origin every anchor consumer resolves through, game and editor alike. ⚠️ These existing to be *imported* rather than copied is not style: the editor drew anchor handles from the sprite's drawn centre while the game resolved the same anchor from a different base, so a handle dragged onto a sprite landed 16px away in game. Always. For every anchor.

Overlay primitives (E-24). `submit_overlay_lines(points_world, color, width, closed)` and `submit_overlay_polys(points_world, color)`: the latter accepts **RGBA**; alpha < 255 alpha-blends via a bounding-box scratch surface. Ellipses are caller-side polygon approximations.

HUD pass. Four frozen, pure, screen-space dataclasses: `HudRect`, `HudText`, `HudSprite`, `HudLines`. The host calls `submit_hud(item)`; at `flush`, after sprites and overlays, they fold into the same flat draw list **in screen space: no coords conversion, no depth sort.** `HudRect` / `HudText` accept RGBA. `HudSprite` animates (`animation`, `anim_time_ms`) with the same slot/animation contract as `RenderItem`.

Nine-slice (A2), HUD only. A `DrawCall` may carry `slice`; only the backend interprets it. Corners blit **1:1, never resampled**; edges stretch on one axis, the centre on both. `clamp_pair` floors negatives to 0 then clamps margins proportionally into source then destination, so **any margins are safe at any destination size**: the editor feeds it unsaved draft margins straight from spinboxes and rendering degrades rather than raising. `transform.scale`, not `smoothscale`: our sheets are pixel art with per-pixel alpha and no `convert_alpha()`.

Backend throughput. (1) A module-level `WeakKeyDictionary` scaled-frame cache keyed by source-surface identity avoids re-running `transform.scale` each frame at zoom ≠ 1; nine-slice composites live in the same dict under a `("9p", size, margins)` key. Flip/tint stay per-call; the grey-X placeholder is a fresh surface each call so it never leaks. (2) Plain sprite draws accumulate into one `target.blits(...)` batch, flushed whenever a non-sprite call must land in order. Both are pixel-transparent.

Ground cache (`render/ground_cache.py`): the panning fix. Windowed culling bounds the ground submit to O(visible), but that is still ~2,600 tile blits + allocations + a full sort **every frame** under the scaled software renderer. `GroundCache` composites the ground layer into an oversized (viewport + 2·margin) surface baked at an anchor pan.

- Steady state (pan unchanged) = **one blit**.
- On pan it **scrolls the surface in place** (`Surface.scroll`, a memmove) and repaints only the newly-exposed edge strip: work proportional to pan *speed*, not viewport area or map size.
- ⚠️ **A thin screen strip is a diagonal in tile space.** An axis-aligned tile window for it balloons to almost the whole viewport. The strip repaint therefore addresses tiles by rotated coords `d = col-row`, `s = col+row` via `tilemap.band_render_items(...)`: ~100 tiles instead of thousands.
- Full rebuild only on first use / zoom / resize / explicit `invalidate()` / a jump clear off the cached surface.
- Renders through a **private** `CoordinateSystem` (a fresh `Camera` at `anchor_pan - margin`), never mutating the host camera. Sub-pixel remainder rides in the blit's float destination, so the surface stays rounding-exact versus a direct render.

Fonts (`render/fonts.py`). A lazy cache keyed by font key: `sm/md/lg/xl/xxl` (lg/xl/xxl bold) plus `hud_phase`, `hud_lvl`. `TextMetrics().size()` gives `(w,h)` without blitting, so pure-metadata code never imports pygame. `configure_fonts(doc, font_path=None)` is the ONE way the specs change after import; it rebinds in place and clears the cache. A cached font whose pygame session was torn down is transparently rebuilt.

> ⚠️ **The custom font file is read once into bytes and every size is built from an `io.BytesIO`, never from the path.** `pygame.font.Font(<path>, size)` makes SDL_ttf hold that file **open** for the font object's lifetime; on Windows that is a hard lock, and the editor sat on the designer's font file for its whole run.

⚠️ **`layout_h` / `_LAYOUT_H` is a pinned constant table and is NEVER touched by `configure_fonts`.** Windows and Linux measure `SysFont(...).size()` heights ±1px apart, so a live measurement baked into a stored rect makes committed artifacts (captured on Windows) diverge from what CI regenerates. Font size changes drawn glyphs only; stored layout rects do not move.

Source. `engine/render/{renderer,backend,item,hud,fonts,ground_cache}.py` · `engine/render/CLAUDE.md`

3.6 `engine/vfx`: procedural effects

> **Owns**: pure particle/decal emitters + the frozen parameter dataclasses they take.

Three-file split:

FILE	CONTENT
`params.py`	frozen dataclasses with **no defaults**, one per `data/balancing/vfx.json` `procedural.*` table. A default here would be a second home for a value that belongs in `data/`.
`particle.py`	the stateful `Particle` / `GoldHighlight` / `Slash` objects
`emitters.py`	pure `emit_*(rng, …)` functions: ⚠️ **every emitter takes an injected `rng`**, never the stdlib `random` module, so seeded-parity tests are possible
`system.py`	`VfxSystem` owns the particle/gold/slash/splatter lists, `update(dt)`, and **two** submit surfaces (world-overlay vs HUD: the host interleaves them at different points in the frame, so one submit method would reorder the draw)
`play_once.py`	`PlayOnceVfx` / `PlayOnceFade` / `spawn_play_once`: the generic one-shot **sprite** VFX a designer's trigger row can bind a `vfx_*` slot to. Returns `None` when the slot has no imported art: the caller's cue to run its procedural fallback.

⚠️ **`VfxSystem` owns state for exactly five families** (spark, death-burst, muzzle, slash, gold highlight, splatter). The beam / crater / lightning / announce / floater / projectile dataclasses are held by `game/ui/effects.py` directly, because the **scene already owns** those fade clocks (`CraterFade`, `LightningFXFade` components). A parallel engine-side list would be a second source of truth for the same fade.

⚠️ **This package never learns a JSON key name.** `game/ui/effects.py`'s `_params_from_balance` is the ONE place a `vfx.json` key and an `engine.vfx` field meet. Do not add a convenience `load_defaults()` helper.

3.7 Top-level engine modules

MODULE	PURE?	OWNS
`tilemap.py`	✅ pure	**The one authority for the map file format (D-20)**, shared by game and editor. `TileMapDoc`, `load_map` / `save_map`, `validate_doc`, and three render emitters.
`data_io.py`	✅ pure	`write_validated` / `load_validated`: schema-validating JSON I/O with deterministic dumps (D-3).
`audio.py`	pygame	Thin `pygame.mixer.music` wrapper. ⚠️ **Every call swallows all exceptions** → silent no-op when audio is unavailable.
`video.py`	lazy cv2	`VideoSource` for cutscenes. cv2 imported lazily; **graceful skip** if absent/missing/unopenable.
`video_playback.py`	✅ pure	The clock/state machine `video.py` composes for `elapsed` / `progress`. Usable and tested standalone.

MODULE	PURE?	OWNS
`tutorial.py`	✅ pure	A generic step-sequencer. **Knows nothing of tiles, buildings, cards or love**: every `Step` field is an opaque string the caller gives meaning to.

`tilemap.py` detail. It carries **no game vocabulary**: terrain cells are single chars resolved through the map file's own schema-pinned `legend`. It schema-validates *and* cross-checks what the schema cannot express (row counts and lengths vs dims, markers in bounds, `id == filename stem`) and fails loud.

Three emitters, pick by need:

EMITTER	USE
`render_items(doc, …)`	the WHOLE map. Tests and small full-map consumers.
`visible_render_items(doc, col_min, col_max, row_min, row_max, …)`	the **windowed** variant. Pair with `visible_tile_window`: this is why a 1024^2 map stays at full fps.
`band_render_items(doc, d_min, d_max, s_min, s_max, …)`	ground only, addressed by rotated iso coords `d = col-row`, `s = col+row`, for the ground cache's diagonal scroll-fill. Takes `code_overrides` so runtime zone state (unlock/recede) shows without mutating the doc.

> ⚠️ **Checkerboard parity is prototype-exact**: a `_b` suffix is appended iff the legend entry has `checker: true` **and** `(col + row + 1) % 2 == 1`. Background kinds never alternate.

> ⚠️ The nullable single-object markers (`camera_start`, `start_area`, `tutorial_flute`, `tutorial_stone`) are **deliberately never rendered by the emitters.** The game does not draw them; the editor draws outlines via `submit_overlay_lines`.

Video pacing. `update(dt)` paces frame reads by the **source's own fps** (probed once at open), not one frame per host call: the host's 60fps no longer dictates playback speed. At most 10 frames are read per call so a debugger stall cannot spin through the clip. ⚠️ `length` no longer ends a live capture: only EOF or an explicit `skip()` does, so the full authored clip always plays.

3.8 The pygame allow-list

pygame imports are permitted **only** in:

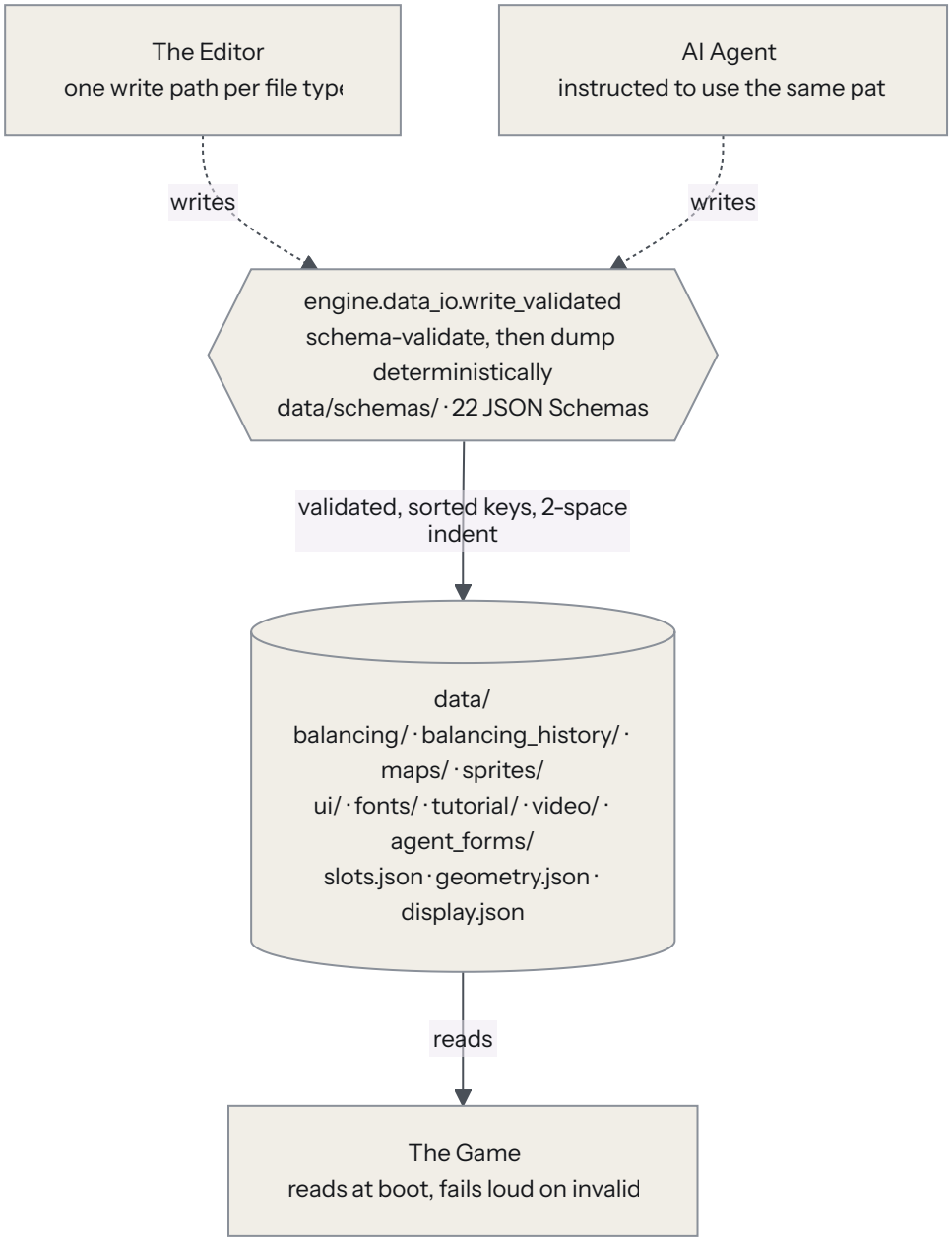
`render/backend.py` · `render/fonts.py` · `render/ground_cache.py` · `assets/store.py` · `assets/placeholder.py` · `engine/audio.py` · `engine/video.py`

Everything else (`coords/`, `core/`, `physics/`, `vfx/`, `tilemap.py`, `data_io.py`, `video_playback.py`, `tutorial.py`, asset *metadata* code) is pure Python. **That is what keeps game logic headless-testable.**

Part 4: The data layer (`data/`)

Requirements: `SPEC.md` §5 (`D-*`). Doc: `data/CLAUDE.md`.

One door, and everything goes through it.



Designers never open these files: the editor is the interface. Agents may edit directly, but only schema-valid writes. The game only ever reads.

4.1 Principles

#	PRINCIPLE
D-1	JSON under `data/` is the **only** value store. No balance or content values in Python. The prototype's py+json dual system is dead; never reintroduce it.
D-2	Every file type has a JSON Schema in `data/schemas/`. Writers validate on save; the game validates on load and **fails loud in dev**.
D-3	Deterministic formatting: sorted keys, 2-space indent, trailing newline. Git diffs stay minimal and agent edits stay cheap.
D-4	Designers never open these files: the editor is the interface. Agents may edit directly, but only schema-valid writes.

Two more rules that bind the test suite:

> ⚠️ **Tests must never write into `data/`.** They copy it to a tempdir (`TempDataCase`). A session fixture hashes `data/` before and after the suite and fails the run if a single byte changed. This is not theoretical: the suite silently painted tiles into real maps and invented map files for months.

> ⚠️ **Never assert against live `data/` content, pin the fixture.** "The Painter slot has no art" and "first_light is the active map" were both true when written and both false later. That is what put 18 tests permanently in the red.

4.2 File inventory

PATH	SCHEMA	WHAT IT IS
`data/geometry.json`	`geometry`	tile pitch, fallback dims, fallback zooms
`data/display.json`	`display`	window size, fps, caption
`data/slots.json`	`slots`	**the slot registry**: which asset slots exist, per category
`data/balancing/{buildings,enemies,map,ui,core,vfx}.json`	`<domain>`	every gameplay tunable
`data/balancing_history/*.json`	`balancing_history`	newest-first full-doc snapshots, appended by the editor's Save
`data/maps/<id>.json`	`map_file` †	terrain grid, deco, base, markers
`data/maps/active_map.json`	`active_map`	`{"active": "<map_id>"}`
`data/sprites/asset_manifest.json`	`asset_manifest`	manifest v2
`data/sprites/imported/*.png`	-	**committed content**, not build artifacts
`data/ui/screens/<id>.json`	`ui_screen` †	per-screen widget overrides
`data/ui/screen_defaults.json`	`screen_defaults`	generated-but-committed default layouts
`data/ui/fonts.json`	`fonts`	the 7 font presets (size + bold)
`data/ui/palette.json`	`palette`	one key per `C_*` UI colour
`data/ui/strings.json`	`strings`	flat `{string_id: template}` UI text table
`data/ui/active_font.json`	`active_font`	pointer to the game-wide custom font

PATH	SCHEMA	WHAT IT IS
`data/fonts/font_manifest.json`	`font_manifest`	imported `.ttf` / `.otf` registry
`data/tutorial/tutorial.json`	`tutorial`	the guided-tutorial script
`data/video/cutscenes.json`	`cutscenes`	cutscene registry keyed by trigger
`data/agent_forms/*.json`	`agent_form` †	the "Add new X" form specs the editor renders
-	`dispatch_handoff`	the only schema with no `data/` content file; handoffs go to gitignored `.claude/dispatch/`

† = a **directory** schema-pairing exception, see §4.3.

4.3 Schema pairing

Default: stem pairing. `data/foo.json` ⇔ `data/schemas/foo.schema.json`. A missing schema fails loud.

Exactly four exceptions are implemented and tested in `tools/smoke.py::validate_data`:

1. every `data/maps/*.json` **except** `active_map.json` → `map_file.schema.json`
2. every `data/balancing_history/*.json` → `balancing_history.schema.json` (its stem deliberately collides with the real domain file's stem)
3. every `data/agent_forms/*.json` → `agent_form.schema.json` (the stem is the form `id`, cross-checked by the loader)
4. every `data/ui/screens/*.json` → `ui_screen.schema.json` (the stem is the screen id)

Adding a fifth means editing that `if/elif` chain **and** pinning it in a test. `data/schemas/` itself is skipped entirely during validation.

Schema house style. `$id`, draft 2020-12, `additionalProperties: false`, all keys `required`, local `#/$defs/` refs only (cross-file refs are forbidden), no `allOf` composition (it breaks `additionalProperties:false`). Every property carries a `description` documenting units and scale (a test enforces presence) and every numeric property declares `minimum` / `maximum`: **the editor derives spinbox ranges from them, which is what makes invalid input unrepresentable (ED-30).**

> ⚠️ **No `oneOf` unions in balancing schemas.** `editor/panels/balancing.py` reads `prop.get("type")`; a type-less node raises `no widget for schema` and crashes the balancing panel for the whole domain.

Bounds policy (documented per-domain in the schema description): fractions and chances 0–1 · HP/DMG (×10) 0–100000 · costs and counts 0–10000 · rounds and levels 0–1000 · seconds 0–60 · pixels ±4096.

4.4 Balancing domains

Six files. The domain list is **derived** by the editor (`slots.json` category order ∩ categories that carry a `data/balancing/<key>.json`), never hardcoded. A new domain therefore appears in the selector and the balancing form with zero editor edits.

DOMAIN	TOP-LEVEL GROUPS
buildings	`BuildingsGlobal` · `DefenceBuildings` (BasicDefence, AOEDefence, BeamDefence, StormPriest, globals) · `EconomyBuildings` (Musicians, Meditators, Painters) · `BoostBuildings` (Speed, Damage, HP, globals) · `StructureBuildings` (Blocker, WallBuilder)

DOMAIN	TOP-LEVEL GROUPS
enemies	`EnemyScaling` · `EnemyTypes` (Standard, Raider, SiegeCannon, Formation, Boss) · `MortarTargeting`
map	`Pathfinding` (content_weights, damage_reduction) · `TileUnlocking` · `TileConditions` (spawn_chances, modifiers, path_weights, min_speed_fraction) · `SpawnDeco`
core	`Camera` · `General` · `PhaseLoop` · `TheHole` · `XP` · `LightningStrike` · `Tutorial`
ui	`Menu` · `Timing` · `FX` · `Debug`
vfx	`procedural` (spark, death_burst, muzzle, slash, gold_highlight, splatter, beam, crater, lightning, announce, floaters, projectile) · `triggers` (10 cosmetic event rows)

Structure convention (the "9A REPLAN tree"). PascalCase group objects, snake_case leaves, tier struct-lists under a `tiers` key. Every object level keeps `additionalProperties:false` + a full `required` list, no exceptions.

Per-enemy-type required leaves worth knowing:

LEAF	MEANING
`footprint`	int 1–8. The unit occupies footprint² tiles; its sprite is downscaled to `footprint × tile_w` wide, **never upscaled**.
`sprite_scale`	0.1–8. Applied **after** the fit: the knob for low-res art.
`registry_group`	the `slots.json` "enemies" group label its sprites live under. Exists so the editor can resolve a preview's real render fit without importing `game/`.
`kidnapping`	bool. Whether a killing blow on a building turns this unit into a carrier.
`death_spawn`	`{at_hp_fraction, enabled, spawn_hp_fraction, spawns[]}`: see §5.8.

> ⚠️ `spawns` **is always an ARRAY of per-era rows**, resolved `spawns[clamp(era, 0, len-1)]`. The Boss carries 5 rows; a type with no eras carries 1 and always clamps to row 0: that *is* the "flat table" case. There is deliberately no union.

The parity gate is gone. The migration is complete: no test compares `data/` against the prototype, and there is no mapping table to mirror. Moving, renaming, retuning or dropping a balancing key is a design decision, not a regression. **The schemas are the only guard rail.**

4.5 Slot registry (`data/slots.json`)

Ordered `categories[]`: the array order **is** the editor tree order.

CATEGORY	FRAME	ANIMATIONS	GROUPS
`buildings`	64×96	idle, attack, death, hurt, place, upgrade	Defender, Musician, Meditator, Maw Mortar, Painter, Sun Scorcher, Storm Priest, Speed/Damage/HP Booster, Blocker, Wall Builder
`enemies`	64×96	idle, walk, attack, death, kidnap	Walker, Raider, Siege Cannon, Formation, Boss
`map`	64×32	idle	Tiles, Engine
`ui`	64×64	**idle, hover, pressed, disabled**	Buttons, Panels, Icons, Backgrounds
`core`	64×96	idle	Base, Camera Start, Start Area, Tutorial Flute, Tutorial Stone
`vfx`	64×64	idle	Effects

CATEGORY	FRAME	ANIMATIONS	GROUPS
`deco`	64×96	idle	Props
`conditions`	64×96	idle	Grass, Mountain, Pond, Forest
`backgrounds`	480×270	idle	Main Menu

`animations[0]` is always `idle` (schema-enforced). `groups` is a recursive tree of `{label, slots[] XOR children[]}`.

Variant families. A leaf group whose slots are *interchangeable art for one thing*: enemy eras, deco prop types, UI skins. The editor's "+ Variant" button appends `<stem>_v<k>`. ⚠️ **This is why every `ui` group is a parent with leaf children rather than a flat slot list**: a flat leaf makes `variant_target()` return `None` and "+ Variant" silently dies.

> ⚠️ `map → Tiles → Background` is **not** a variant family: every background slot needs its own map-file legend code, so "another background" is another numbered `tile_background_<n>` type.

A slot key may repeat across groups of one category (shared art) but never across categories: the frame size would be ambiguous, and the loader rejects it.

A sheet may be shared. `sheet` is a path, not a slot-derived name. The editor's "Use Spritesheet…" points a slot's entry at another slot's PNG and copies no bytes. ⚠️ **Deleting art must refcount** (`sheet_users` / `unreferenced_sheets`): a PNG is unlinked only when no remaining entry points at it. ⚠️ **Orphans are legal and deliberate**: re-linking a slot away from art only it used leaves that PNG on disk and listed in the picker, which is how you get it back.

4.6 Asset manifest v2

```
{ "version": 2,
  "entries": {
    "<slot_key>": {
      "sheet": "imported/<name>.png",
      "frame_w": 64, "frame_h": 96,
      "offset_x": 0, "offset_y": 0,
      "rows": [ { "animation": "idle", "frames": 4, "fps": 8,
                  "hidden": [], "loop_start": 0, "loop_end": 3,
                  "loop_count": 1 } ],
      "slice": [4,4,4,4],            // optional, HUD nine-slice
      "anchors": { "muzzle": [12,-30] },  // optional, metadata only
      "tint_overlay": false          // optional, condition art only
    }
  }
}
```

Written **only** by the editor's import panel, through `write_validated`. The one-shot migration tool that seeded it is deleted: the editor is the only door.

4.7 Map files

```
id            == filename stem (loader-enforced)
display_name
cols, rows    4…1024, each map owns its own dims
terrain       rows[] of cols-length single-char code strings (one line per row → cheap diffs)
legend        schema-pinned char → {slot, checker} table
              b/c/s = buildable / combat / spawning   (checkerboard _b alternation)
              f/l/o = forest / cliff / ocean          (background, no alternation)
base          {col, row, slot}   slot const-pinned to base_hole
camera_start  nullable {col,row,slot}
start_area    nullable {col,row,slot}:  the 2×2 starting area's MIN corner
tutorial_flute / tutorial_stone  nullable {col,row,slot}
deco          [{slot, wx, wy, flip?}]:  renders ABOVE entities
```

Spawning is a painted tile zone, not point objects. The format has no spawn-point objects; enemies enter from spawning-zone tiles.

The file is self-describing. Because the legend lives in the file, no package hardcodes tile vocabulary. Dimension consistency is beyond JSON Schema, so `engine.tilemap.load_map` cross-checks and fails loud.

`maps/first_light.json` is the committed starter map (prototype-exact initial layout) so the game always boots on real data.

4.8 UI & theme data

Per-screen overrides (`data/ui/screens/<id>.json`): 14 files, one per screen: main_menu, pause, settings, credits, add_name, game_over, levelup, hud, building_panel, cheat_menu, game_log, boss_cutscene, overlays, tutorial_message.

```
background: {slot} | {color}
defaults:   {button_skin?, panel_skin?, font?, text_color?}   ← for DYNAMIC-count
                                                                content with no id
widgets:    {<id>: {rect?, skin?, tint?, font?, color?, text_color?,
                    label?, visible?}}                        ← per named widget
```

A widget's override is always a **partial patch**, never a full record: the object carries no `required` array, so absent-by-default is the rule and an old screen doc that predates a new key keeps validating unchanged.

Generated defaults (`data/ui/screen_defaults.json`): written by `tools/export_ui_layouts.py`, committed, and validated by a test that re-runs the exporter. Flat, keyed by screen id: `{<id>: {widgets: {<id>: {rect, kind, label}}, mock_note}}` where `kind ∈ {button, panel, label, backdrop, bar, field}`: a pinned six-value enum. Merge conflicts resolve by re-running the exporter.

Theme data. `fonts.json` (exactly the 7 preset keys), `palette.json` (one key per `C_*` colour), `strings.json` (flat dotted-id → template map), `active_font.json` (pointer). All four are loaded and validated at boot and **fail loud**: they are config data, not art, so the E-37 tolerance does not apply.

> ⚠️ **Parity is the safety net.** The committed content is today's hardcoded values verbatim, and `test_theme_data.py` pins that (a) configuring from the fixture reproduces the golden UI baseline byte-for-byte and (b) the *unconfigured* module defaults equal that same fixture. The two value sets can never silently drift.

4.9 Tutorial & cutscenes

`data/tutorial/tutorial.json`: the guided script.

```
skippable, first_loss_costs_life          behavioural toggles
messages   { economy_intro, lives_intro, close_panel_hint }   ← a CLOSED 3-key object
steps[]    { id, message?, highlight[], advance_on, allow[],
             flags{}, revert_on?, revert_to? }
```

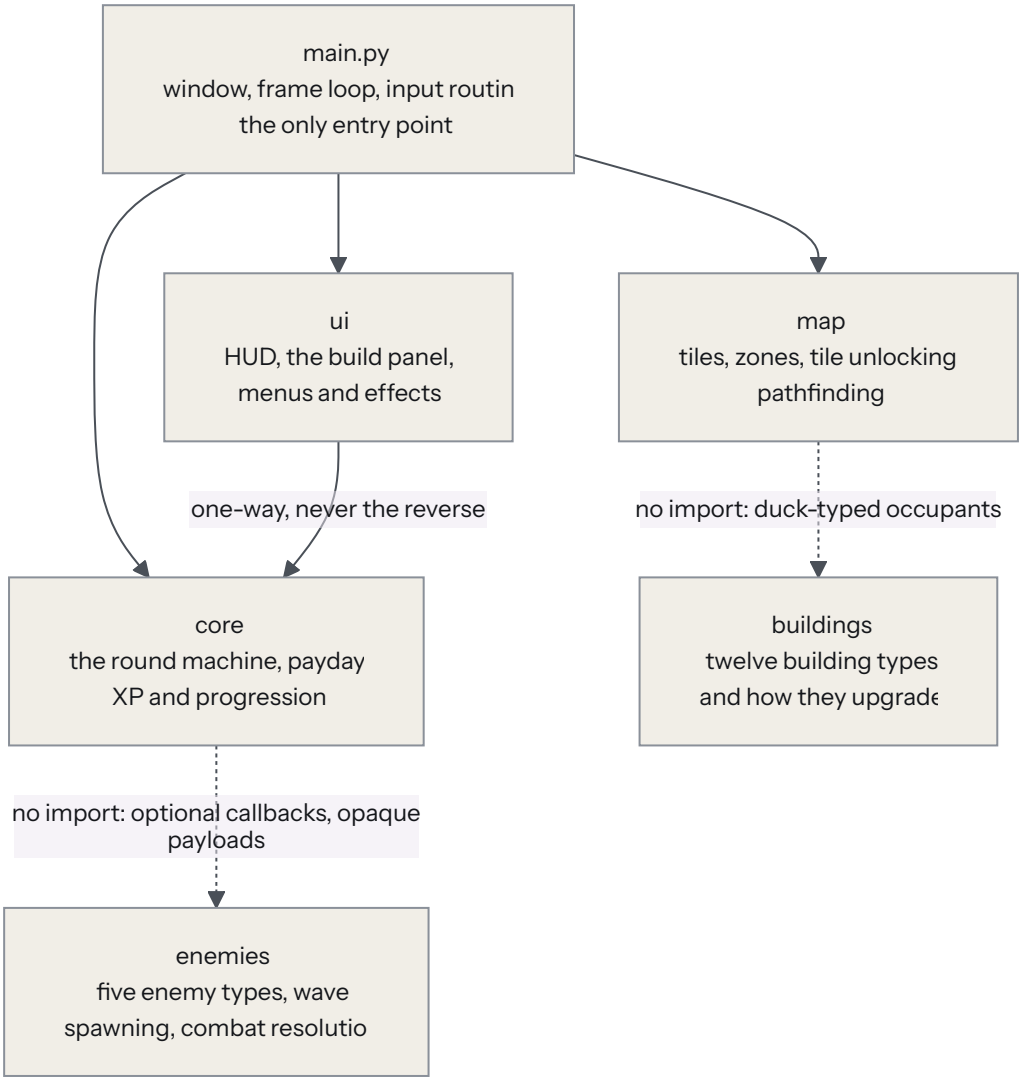
Every step field is an **opaque string** the game-side director binds to a real thing. `flags` is the one deliberately open leaf (`additionalProperties: true`) so later phases attach per-step data with no schema bump.

`data/video/cutscenes.json` : an open registry keyed by cutscene id, each `{video, audio?, length, trigger}` . `trigger` is a closed enum today (`intro` | `first_end_turn`); a new trigger point is a schema bump.

Part 5: The game (`game/`)

Requirements: `SPEC.md` §6 (`G-*`). Router: `game/CLAUDE.md`. Five domain folders mirror the five original balancing domains and still scope file ownership.

The five domain folders, and the seams between them. The dotted links are deliberately *not* imports: they are the mechanisms that let two folders cooperate without one depending on the other.



5.1 Host & frame order

`game/main.py` is the **only** entry point.

```
main(max_frames=None, data_dir=None, autostart=False)
```

- `max_frames` makes it importable so `tools/smoke.py` drives the same code headlessly **(G-8)**.
- `autostart=True` skips the shell straight into GAMEPLAY: the headless seam that keeps boot tests exercising the full `_World` / `Session` construction the menu would otherwise defer.

Frame order is fixed (E-14): input → `Scene.update(dt)` → render submit → flush → flip.

Camera input mapping lives here, on pure engine camera state:

INPUT	EFFECT
Left-drag (began over the world, past a 4px threshold)	pan
Right-drag (past 4px)	pan
Left-click (under 4px)	select / place
Right-click (under 4px)	**universal dismiss**: from anywhere on screen, panel and HUD included. Closes the cheat menu, else peels one stage off the building panel and clears multi-select. LEVELUP / boss cutscene are choice-only and swallow it.
Scroll wheel	steps through `zoom_levels`, keeping the viewport-centre world point fixed via `screen_to_world` / `world_to_screen` only
`Esc`	pause (gameplay) / back out (menus)
`1` `2` `3`	combat speed 1× / 1.5× / 2×: ENEMY phase only, round-gated
`P`	quick-skip the wave
`Ctrl+L`	cheat menu (a deliberate divergence from the prototype's `Ctrl+P`, since bare `P` is taken)

> ⚠️ The 4px threshold is what keeps a click and a drag apart on **both** buttons.

Session wiring, per frame.

```
session.pre_sim(sim_dt, scene)
scene.update(sim_dt)
resolve_combat(scene, tilemap, sim_dt, buildings_balance,
               on_base_hit=…, on_enemy_death=…, on_kidnap=…,
               on_splash_impact=…, on_defender_fire=…, on_projectile_hit=…,
               dmg_bonus=…)
session.post_sim(scene)
```

Combat speed is a host concern. `Session` owns the selector and the round gates; `main.py` owns where it lands: `sim_dt = dt * combat_speed` while `phase == ENEMY`, else plain `dt`. ⚠️ **Never scale the ROUND_END/INCOME timers**: a 2× wave still gets its full payday beat.

Screen shake (boss rounds) is a transient `cs.pan(ox, oy)` / `cs.pan(-ox, -oy)` wrap around the world render branch, with **no clamp between**.

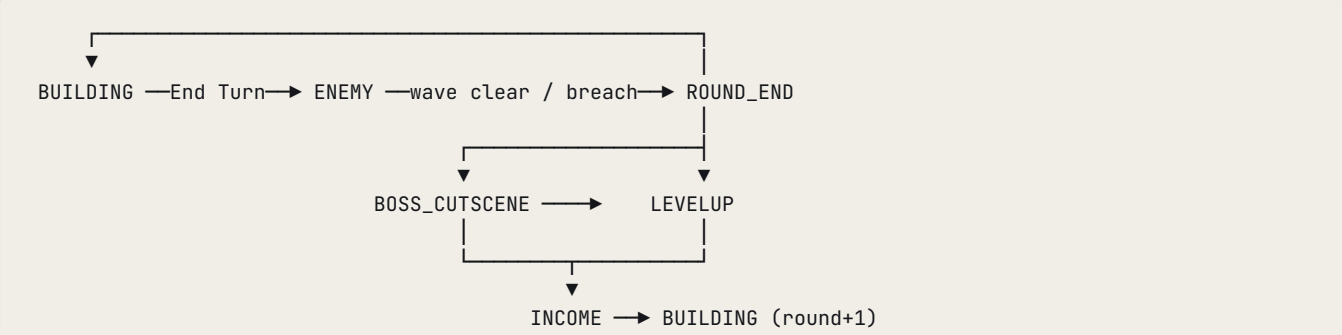
5.2 State machines

Top level: `GameState` (owned by `game/ui/shell.py`, which is **pure**):

```
CUTSCENE ──▶ MAIN_MENU ──▶ GAMEPLAY ──▶ GAME_OVER ──▶ MAIN_MENU
                  │  ▲          │ ▲
                  │  │          ▼ │
                  │  └──────── PAUSED
                  ├──▶ SETTINGS ─┘        (one SETTINGS state, reused from both
                  ├──▶ CREDITS            entry points via `settings_caller`:
                  └──▶ ADD_NAME           there is deliberately no SETTINGS_PAUSED)
```

`Shell` applies pure transitions itself and returns an **intent string** only for host-side (pygame/disk) actions: `new_game` · `quit_to_menu` · `quit_app` · `set_display_mode` · `add_name_commit`. The host executes intents and owns window re-creation, the cutscene blit, music, and the `_World` lifecycle. **Menus hold no world**; a fresh `_World` *is* a fresh run.

In-round: `GamePhase`:



Precedence at ROUND_END expiry: a pending boss cutscene **beats** a pending level-up; the cutscene chains → LEVELUP (if pending) → payday, exactly once. `Session.frozen` is `phase in (LEVELUP, BOSS_CUTSCENE)`: the single "skip the whole sim" flag.

> ⚠️ Everything freezes on `GAME_OVER` (no phase advances): the prototype's `_update` has no GAME_OVER branch and this matches it.

Wave-clear condition (`Session.post_sim`): four terms, each added for a real bug:

```
spawner.done
  and no live "enemy"
  and not scene.queued_by_tag("enemy")     ← children burst THIS frame (ER-5)
  and not scene.by_tag("kidnapper")        ← a carrier walking home HOLDS the round open
  and not scene.queued_by_tag("kidnapper")
```

> ⚠️ **Every "clear the field" path must clear kidnappers too** (`quick_skip_combat`, `_wipe_round`, `cheat_skip_round`, `cheat_goto_round`), or the round can never end once one exists.

5.3 Payday: the twelve-step settlement

`game/core/payday.py::run_payday`. **The ordering is SACROSANCT. Do not reorder without the user.**

#	STEP	WHY HERE
1	Reset income floaters	the per-tile ledger the UI reads
2	Snapshot `RoundStats` (this-round → last-round, then zero)	everything downstream reads *last* round
3	Boss story payouts (Boss1B / Boss3B)	**after** the snapshot: Boss3B reads the `dmg_dealt_last_round` it just rolled. Paid silently, no floater.
4	Base income + `yield_amount` sweep	base income is village-level-scaled; Boss2A/2B per-recipient deltas fold in *here* so floaters, totals and the HUD stay in lockstep
5	Upkeep sweep (clamped at 0)	mirror of income
6	**Painter** payout / tile release	⚠️ **before revive**: must see a dead painter as `alive == False`
7	**Boost** accumulate / explode-on-death	⚠️ **before revive**, same reason

#	STEP	WHY HERE
8	**Wall teardown** for dead wall-builders	⚠️ **before revive**, same reason
9	**Revive / heal**: every non-base building full-heals (base excluded)	gated by `core.TheHole.building_revive`
10	**Rebuild walls**: every alive builder restores its frozen snapshot at full HP	after revive, so a revived builder's walls come back
11	`round_num += 1`	
12	`phase = INCOME`, start the payday-floater timer	

Slots 6/7/8 sitting before 9 is the entire reason the ordering is called out. A building killed this round must be *seen dead* by the painter/boost/wall logic before the revive sweep resurrects it.

The one non-duck-typed branch is in step 4: a Meditator is asked `collect_income(disturbed)` instead of `yield_amount()`, because its streak compounding has a side effect. ⚠️ `yield_amount()` itself is **pure**: three callers read it (payday, the panel, the HUD). Do not move the side effect back in.

5.4 Economy & progression

Love. `RunState` is the single owner. `add_love` / `spend_love` clamp at ≥ 0 (the prototype clamps every currency write).

Income sources, in the order the HUD tooltip lists them: Base (village-scaled) · Musicians · Meditators · Story (boss bonuses) · − Upkeep. ⚠️ `hud.income_sources(session)` is the ordered list and `income_breakdown` sums it, so the pill and the tooltip cannot drift.

XP → village level. `game/core/xp.py` (pure):

FUNCTION	ROLE
`xp_for_etype`	keyed on `Enemy.ETYPE`
`award_xp`	arms `levelup_pending`, queues an `xp_events` floater
`advance_village_level`	walks the rising threshold; surplus carries forward, **one level per resolve**
`scaled_base_income`	the ONE source for payday, the HUD income line, and the base-info panel

XP award sites (all via callbacks, so `game/core` imports no `game/enemies`): field kills · a kidnap transition · base-damage kills (gated) · **queued-but-never-spawned enemies on a lives wipe** · building deaths (gated, **once per building `id()` for the whole run**, a faithful prototype quirk: revive, die again, no second payout).

> ⚠️ `_award_building_deaths` runs from `pre_sim`'s ENEMY arm **and** from both of `post_sim`'s round-ending branches. The second site is not redundant: a building that dies on the very frame the round ends never sees another ENEMY-phase `pre_sim`, and payday's revive then makes it `alive` again: its XP was silently lost forever.

> ⚠️ **Quick-skip pays NO XP** (neither cleared nor queued enemies): prototype-exact, and deliberately unlike a lives-breach wipe, which *does* pay the queued enemies so a life loss cannot rob the player.

Research / gating. `game/buildings/research.py` is the extension seam: `LEAF_CLASSES` + a `RESEARCH` table of `ResearchSpec` rows. A spec never stores a gate **value**, only *where in `buildings.json` to read it*.

- **Whether a type starts unlocked is DATA**, read live off the leaf's own group node (`<group>.starts_unlocked`). Only `defence` (Stone Thrower) and `economic` (Flute Player) start unlocked; every other type is earned via a level-up

card.

- **There is exactly ONE round gate per type:** `tiers[0].unlock_min_round`**.** It gates the type's unlock card. A locked type never shows a tier card, so tier 0's round doubles as the type's era gate.
- **Unlocking a type makes its tier 1 immediately placeable.** (An earlier `starts_with_tier` flag caused a double-unlock-card bug and is deleted.)
- Only the **single next** locked tier is ever offerable. Research is **global per type**.
- `upgrade_gate` is the five-mode classifier the panel renders: `in_tier` · `tier_upgrade` · `tier_locked` · `tier_hidden` · `max_tier`.
- **Grouped unlock**: the boost trio surfaces as ONE "Unlock Boost Buildings" card (offered only for the lead member of a spec's `unlock_group`); applying it unlocks all three.
- ⚠️ **An empty level-up pool is expected before round 10**: the roll pads to three, so early level-ups show three identical `+25 Love` cards.

One price per tier. `build_cost(type, balance, tier_idx)` is charged identically for a fresh placement, the level-up tier research card, and the upgrade panel's advance button. A fresh placement builds at the type's **current research ceiling**, not always tier 0.

5.5 Map runtime

`game/map/` is a **runtime layer over an** `engine.tilemap.TileMapDoc`: it never re-parses map JSON.

Zones seed from terrain codes, not procedural rings: `b`→BUILDABLE, `c`→COMBAT, `s`→SPAWNING, `f/l/o`→BACKGROUND, `doc.base` tile→BUILT.

Anchoring. When the map carries a `start_area` marker, the 2×2 unlock/section grid anchors at its min corner: the marker **is** section (0,0). The marker never forces tile states (painted terrain wins; the editor warns if its 4 cells are not buildable). Maps without a marker fall back to legacy base anchoring.

Unlock cost is direction-agnostic:

```
base_unlock_cost + max(0, manhattan_section_distance - 1) * unlock_cost_distance_mod
```

Sections adjacent to the start cost exactly the base cost. (The old signed formula went negative left/above the start.)

Zone changes show on the ground without mutating the doc. `set_tile_state` records the new zone code in `TileMap.terrain_overrides` and fires the host-wired `on_zone_change`. `main.py` feeds the overrides to `band_render_items` and wires `on_zone_change = ground_cache.invalidate`. A new game builds a fresh `TileMap` → empty overrides → pristine terrain.

Spawn recede is dual-axis and backfills strictly behind. A successful unlock converts the nearest SPAWNING 2×2 row-aligned with the bought chunk **and** the nearest col-aligned one to COMBAT (an axis with no aligned band is skipped: there is no nearest-overall fallback); each converted block then backfills the nearest BACKGROUND 2×2 strictly behind it. Nothing behind (map edge) → no backfill, the band shrinks by one block. Both conversions happen before either backfill so the second axis cannot re-find the first's block.

Tile conditions roll **once** at `TileMap.__init__` per `TileConditions.spawn_chances`, for every tile except BACKGROUND-at-init and the starting unlocked pocket (both stay GRASS forever, prototype-exact). ⚠️ `rng` is both the on-switch and the determinism seam: the host passes module `random` (live) or `random.Random(seed)` (tests); `rng=None` **skips the roll entirely**: the all-GRASS fixture mode every path-cost test relies on.

Condition art is state-driven. 16 slots: `cond_<condition>_<state>` for the four conditions × four zone states. A tile picks a stable `condition_variant_idx` once; ⚠️ `set_tile_state` **re-resolves the art slot on every transition** at that same variant index against the new state's family: so a mountain looks different once built over, and a tile's art switches

live as its zone changes.

`game/map/conditions.py` is the ONE emitter, on the **`terrain`** layer, windowed. ⚠️ An un-imported condition emits **nothing** rather than a grey X; the flat colour diamond fallback is drawn by `game/ui/overlays.py` for exactly those tiles, both reading the same `{slot: tint_overlay}` map so sprite and tint can never disagree about what exists.

Spawn-band tree deco rides the *same* init pass (deliberately: a third O(map) walk is the perf invariant this avoids). One packed int per tile (`-1` = no tree, else `variant_idx * 2 + flip_bit`). ⚠️ `spawn_deco.spawn_tree_slots(registry)` is the ONE family definition and both consumers must go through it: the emitter re-bases an out-of-range index with `% len` , so a disagreement would silently **skew the variant distribution** instead of failing.

> ⚠️ Unlike condition art, the tree emitter reads `tile.state` **live at emit time**, so a SPAWNING→COMBAT conversion stops it being emitted the next frame with no `set_tile_state` hook at all.

Edge walls. `WallEdge` (order-independent edge key) + a `TileMap.wall_edges` registry. A WallBuilder's `on_placed` walls only the **outermost perimeter** (player tile edges facing an exterior-combat tile, found by BFS from the spawn zone through COMBAT/SPAWNING only), frozen into the builder's snapshot. ⚠️ The map layer stays import-free of `game.buildings` : it **duck-types** the builder.

Occupancy is occupant-driven and updated **incrementally** (`occupancy.set` per placed tile). `sync_occupancy` is the full-grid rebuild variant and is **never on the placement path**.

5.6 Pathfinding

`game/map/pathfinder.py` .

Base pathfinding is a shared flow field. `find_path` and `find_path_ignoring_walls` walk ONE cached reverse-Dijkstra field seeded at the base, rather than a Dijkstra per enemy spawn. Reverse edges cost the weight of the *block a forward walker would enter*, so field distances equal forward costs exactly.

> ⚠️ **The invariant: ONE Dijkstra per topology change PER FOOTPRINT, never one per enemy.** The cache is keyed on `TileMap._path_version` × `(ignore_walls, footprint)` . **Every weight or blocking mutation must bump `_path_version`** : `set_tile_state` , `set_tile_content` (the ONE occupant/content-key seam: never write `tile.occupant` from outside the map layer), wall add/remove/death, and the two pre-query weight producers (which change-detect their flag sets and bump only on a real difference). A missed bump serves stale paths.

Weight composition is prototype-exact:

```
base weight (from content_weights, keyed by the tile's content key)
  + condition modifier (TileConditions.path_weights)
  + defence-range coverage add
  × damage discount
all gated 0 < w < impassable_weight
```

> ⚠️ Weight is **content-key driven, not `isinstance(Building)`** . Empty tiles derive their key from their zone; occupied tiles carry the key set at placement.

Goal-set variants stay fresh Dijkstras (they are rare):

QUERY	USED BY
`find_path_to_nearest_non_base_building`	the Boss
`find_path_to_nearest_spawn`	a kidnapper walking home (`ignore_walls=True` : a carrier is inert and must never be trappable)

QUERY	USED BY
`find_path_to_nearest_economic` / `_defence`	**dormant**: raider/siege prey-hunting is deferred

⚠️ **`find_path_to_nearest_building` cannot serve the boss**: its predicate is `lambda b: True`, so the base is in the goal set, and `content_weights.base_building` is **0**, cheaper than any real building, so the search walked the boss past its prey and parked it on the hole.

⚠️ **`_dijkstra` keeps a separate tentative `best` map, and that is load-bearing.** It used to guard relaxation on the *settled* `dist` map, where `dist.get(node)` is `inf` for anything unsettled, so *every* relaxation passed and a later, worse one overwrote `prev` with a worse parent. The goal still settled at the right cost (the heap pops in order) but reconstruction walked clobbered back-pointers and returned a route that was **not** the one Dijkstra costed (measured: a 23-cost path to a goal already reached at cost 12, doubling back through a pond).

Footprints: the N×N block convention (ER-2).

RULE	DETAIL
Anchor = MIN corner	a size-N unit at `(c,r)` occupies `{(c+i, r+j) | 0 ≤ i,j < N}`: the body extends right and down. The only convention that works for an even N (no centre tile) and keeps every coordinate an integer.
Passable	every block tile in bounds and under `impassable_weight`, and (unless `ignore_walls`) no live wall on an **internal** edge
A step	additionally clears the whole leading **face** (N edges, not one). `face_edges` is symmetric in its two anchors so the forward Dijkstra and the reverse flow field share it.
Entering a block costs `max(weight)` under the body	a 2×2 avoids a pond even if only one of its four tiles is the pond
A goal is reached when the block COVERS it	not when the anchor sits on it. A 2×2 beside the hole *is* on the hole.

⚠️ **N = 1 collapses every rule to its pre-ER-2 expression, and that is a PERF contract, not just a semantic one.** `_dijkstra` and `_build_flow_field` hoist their loop invariants and take an inline `single = footprint == 1` branch. Calling the block helpers per node/edge instead made a 300×300 rebuild **2.1× slower at footprint 1**. Do not "simplify" that branch away.

⚠️ **Seeding is multi-source and `best` must be pre-seeded to 0.** For N>1 the covering anchors are 4-adjacent, so without it the first seed popped relaxes its siblings and writes a back-pointer into itself, and a unit already covering the base walks on to the lex-min covering anchor instead of stopping.

⚠️ **Footprints never enter `TileOccupancy`.** They are a pathfinding property only: enemies do not block each other, and two footprint-2 units may overlap. That is intended.

5.7 Buildings

`Building(GameObject)` hierarchy. **All state in components.** Derived values are **computed methods on the parents, never stored**: `max_hp`, `upgrade_cost`, `level`, `yield_amount`, `damage`, `upkeep`, `range_tiles`, `attack_speed`. Formulas are `base + (level_in_tier - 1) * per_level`. ⚠️ **Every `upgrade()` / `advance_tier()` full-heals.**

Leaf classes are ≤ ~10 lines: a `SUBTREE` path into `buildings.json`, a `BUILDING_TYPE`, and `TIER_SPRITES` prefixes.

The catalog.

TYPE KEY	FAMILY	BEHAVIOUR
`economic`	Musician (Flute Player)	the baseline earner. **Starts unlocked.**
`meditator`	Meditator	compounding streak. `yield_amount()` is pure; the streak side effect lives in `collect_income(disturbed)`, called only by payday, deriving `disturbed` from `dmg_taken_last_round`.
`painter`	Painter	risky lump sum. `yield_amount()` is `0` (out of both sweeps). On completion it pays, **frees the tile, and permanently bars it** (`used_painter_tiles`). Dying gone-for-good frees the tile without barring it, with a "painting lost!" message.
`defence`	Defender (Stone Thrower)	plain homing projectile. **Starts unlocked.**
`aoe_defence`	Maw Mortar	`SplashAttacker` marker → arcing shell to a **fixed ground point** via predictive lead; full damage in `splash_radius()` with **no falloff and no target cap**; spawns a cosmetic crater. Own `CONTENT_KEY = "aoe_defence_building"` (its own path weight).
`sun_scorcher`	Sun Scorcher	`BeamAttacker` marker → instant hitscan, **highest-HP** targeting, per-tick damage ramp reset on any target change, and a re-acquire pause after a kill. Ticks at its own `BEAM_MIN_TICK = 0.02` floor.
`storm_priest`	Storm Priest	**not a combatant.** `EXTRA_TAGS = ("lightning_source",)` deliberately **drops** the inherited `"combat"` tag, so it is invisible to the combat sweep. It is the Lightning Strike ability's vehicle. **Run-singleton**, enforced in the UI.
`boost_speed` / `boost_damage` / `boost_hp`	Boost trio	ONE behaviour class, three data lines. All buff state lives on the **neighbour's** `BoostReceiver` component; the booster only pushes deltas. Cardinal-4 adjacency. Ramp mode (default) accumulates each payday; flat mode applies 10× once on placement and reverses on death. A booster dead this round **explodes a debuff** onto neighbours once.
`blocker`	Blocker	a pure tier-HP soak. No new enemy code: the standard block-and-attack handles it.
`wall_builder`	Wall Builder	places edge walls on the outermost perimeter, frozen into a snapshot. `wall_hp()` is **not** ×10.
`base`	The Hole	untiered. HP from `core.TheHole.base_hp` = **10, the deliberate not-×10 exception**. ⚠️ Carries **no** `SpriteAnimator`: its sprite is the static map render, so attaching one would double-draw.

Capability by component/tag, never by class. The prototype's `IS_COMBAT` flag is replaced by an `Attacker` component + the `"combat"` scene tag, so the combat sweep stays type-agnostic. ⚠️ Firing *behaviour* lives in the combat sweep, not on the building: the building must not import `game/enemies` (that closes a cycle).

Tile conditions: snapshot at placement, computed on read. `place_building` stamps `_tile_condition` and `_condition_mods` transients, then re-applies stats. ⚠️ **Defence formula order is prototype-exact: boost → condition → explosion debuffs → floor.**

> ⚠️ **Raw vs effective range is a three-way split, and one leg is a deliberate prototype bug kept for parity:**

METHOD	FEEDS
`range_tiles()`: RAW	pathfinding coverage + the RANGE overlay
`effective_range_tiles()`: + mountain bonus	the panel Range row + the selection highlight
`targeting_range_tiles()`	the combat sweep + `RangeSensor`: effective for basic/beam, but **RAW for the mortar**

That last inconsistency is the prototype's own. **Parity audits must not "fix" it.**

Placement seam. `registry.place_building(tilemap, tile, type, love, …)` : buildable-tile + affordability gate → set `tile.occupant` / `content_key` / `state` → `scene.spawn` → incremental occupancy → `building.on_placed(tilemap)` (unconditional base no-op hook) → raise `PlacementError` on a bad tile or too little love.

> ⚠️ Two helpers are **tag-gated, not type-string-gated**, so the registry stays type-agnostic: `lightning.unlock_from_placement` (any `"lightning_source"` building takes lightning 0→1) and `lightning.sync_level_from_tier` (advancing that building's own tier raises the lightning level, latched).

> ⚠️ `game/buildings/storm_priest.py` imports `LightningCaster` **lazily inside** `_extra_components` : a module-level import closes a real cycle.

5.8 Enemies

`Enemy(GameObject)` walker + `Spawner` + a type-agnostic combat sweep.

The catalog.

TYPE	FOOTPRINT	SCALES WITH TIER?	KIDNAPS?	BEHAVIOUR
`Standard` (Walker)	1	✅	✅	paths to the base, attacks whatever blocks it
`Raider`	1	❌ **deliberately**: a glass cannon that only ever grows in *count*	✅	as Standard (prey-hunting deferred)
`SiegeCannon`	1	✅	❌	splits into a **lead group** that heads the queue and a remainder mixed into the shuffled body: cannons open the wave, then trickle
`Formation`	1 (2×2 conceptually via `death_spawn`)	✅	❌	the marching column. **Adds no mechanism**: it is the first consumer of per-slot frame size, footprint pathing, and `death_spawn`, all driven purely from data.
`Boss`	2	❌: reads a per-era stat table	❌	hunts buildings, hole **last**

> ⚠️ **A subclass MUST override `_resolve_stats`, and that is a trap, not a style choice.** The base reads `balance["EnemyTypes"]["Standard"]` **literally**; `STAT_SUBTREE` drives the block lookup in `__init__` but *not* `_resolve_stats`. An un-overridden subclass silently ships walker stats: a bug with no symptom but wrong numbers.

Composition (`Spawner._compose`), checked in this order:

```
round 0?  ──► exactly EnemyScaling.tutorial_round_enemy_count "standard" walkers,
              ignoring every other rule.
              ⚠️ Checked FIRST: 0 % interval == 0 for every interval, so an
              unguarded boss check would treat round 0 as a boss round, and
              (0-1)//n goes negative.
boss round?──► [ONE boss] + ALL siege + shuffle(standard + raiders).
              No siege lead/mix split. Formations never spawn on a boss round:
              deliberate, see below.
else      ──► siege_front + shuffle(standard + raiders + siege_mixed + formations)
```

Counts: `base_enemy_count + (round-1)*(enemies_per_round + tier)` for standard; accretion formulas for raiders, siege and formations. `uniform(0.4, 1.6)` spawn jitter. ⚠️ `_formation_group` is called **last** among the composition groups so every earlier group's rng draw sequence stays byte-identical: the deterministic wave fixtures depend on it.

> ⚠️ **Formations never spawn on a boss round, do not "fix" it.** `_boss_round` composes from a `$defs/spawn_counts` table **shared with every `death_spawn.spawns` row**; adding a `"formation"` key would force a meaningless formation count into every death-spawn row.

Spawn tile clearance. `_pick_spawn_tile` is the ONE choke point all composition sites route through: a footprint-N enemy only spawns where its whole N×N block is spawn zone, cached once per round per footprint. No qualifying tile → **unfiltered fallback**, so an enemy is never dropped from a wave. ⚠️ The filter consumes **zero rng**, and `footprint == 1` takes the byte-identical single `rng.choice` draw, which is what keeps the deterministic composition fixtures green.

Navigation (`PathAgent` component). ⚠️ It runs **before** `Movement` in the component list so its halt decision takes effect the same frame. It gates locomotion by zeroing `Movement.speed` while blocked and restoring it on unblock: **the path is never discarded**, so no re-path is needed when the blocker dies.

Flags, all default-off so the four non-boss types stay byte-identical: `goal_is_base` · `repath_on_kill` · `target_col` / `target_row` (the committed victim, `-1` = none) · `carrying` (a carrier is inert: `update()` returns immediately at the top).

> ⚠️ **Choose by DISTANCE, route by COST.** The victim is picked by plain geometric distance (what the player sees) while the route stays the weighted Dijkstra, so the unit still walks around ponds. One weighted search cannot do both jobs: terrain, defence coverage and the damage discount all bend the cost field, so the cost-nearest building can be across the map from the player's "nearest".

> ⚠️ **`_repath` does not rewind.** It snaps the start to `round(wx)`, but the body is somewhere *inside* that tile: aiming at `path[0]` walked the unit **backward** to the tile centre before setting off (measured: col 11.000 → 10.705 in the second after a kill). It starts at `index = 1` whenever the path has one.

`death_spawn`: the ONE death-spawn mechanic. Required on every enemy type.

```
at_hp_fraction     the unit is dead once hp ≤ max_hp * this.   0.0 = die at zero.
enabled            false = dies normally, spawns nothing.
spawn_hp_fraction  children spawn at this fraction of their OWN max HP.
spawns[]           per-era rows, resolved spawns[clamp(era)].
```

> ⚠️ **"Breaking formation IS dying."** `Enemy.alive` is `hp > max_hp * at_hp_fraction`, and that is the ONE evaluation site. A Formation that scatters at half health just ships `at_hp_fraction: 0.5`; from there the existing pipeline runs untouched. There is **no separate break state and no second state machine.** At `0.0` this is exactly `not Health.is_dead`, so every pre-feature type is byte-identical.

> ⚠️ **Footgun, with no runtime guard by design:** a `spawn_hp_fraction` at or below a child type's own `at_hp_fraction` makes the children die on the frame they appear, and chain, if that child also bursts. Data is the source of truth; the editor's 0..1 spinbox bounds are the fence, and the schema description says so.

The layering handshake. `game/core` imports nothing from `game/enemies`. The session duck-types `death_spawn_plan` / `death_spawned` / `mark_death_spawned()` off *any* enemy and hands the plan straight back to `Spawner.spawn_death_swarm(scene, col, row, plan)`. ⚠️ What crosses the boundary is an **opaque plan dict**, not `(col, row, era)`: core never indexes into it, which makes the layering structurally impossible to violate rather than merely conventional. The stash is a **list**, not a single slot: several units can die in one frame.

Corpse (`corpse.py`). The enemy's death path is byte-identical: it still despawns the frame it dies. To let a `death` animation play, the **host** additionally spawns a cosmetic `Corpse` tagged `"corpse"` (never `"enemy"`), carrying only a `SpriteAnimator` + a `CorpseFade` clock. It is invisible to every gameplay query. ⚠️ Its lifetime is the manifest `death` track's `total_ms`, and `animation_total_ms` returns `None` (not the idle duration) when there is no `death` row, so a sheet without one keeps the instant despawn. Only real field deaths get a corpse.

Kidnapping (`kidnap.py`). A kidnap-capable enemy's killing blow on a building is **terminal for the carrier, ordinary for the building**.

> ⚠️ **The retag trick.** On transition, the owner's tags flip `("enemy", …) → ("kidnapper",)`. `GameObject.tags` sits in `_ENGINE_ATTRS`, so the write is legal past the setattr seal. **One move** makes the carrier invisible to *every* query that reads `by_tag("enemy")` (the combat sweep, the beam's sticky target, base-arrival resolution, lightning, the overhead HP bars, the heatmap tracker) with **no per-site "if kidnapping" filter anywhere.** Consequence to accept: a damaged kidnapper shows no HP bar (moot: combat can no longer see it).

> ⚠️ `pa.goal_is_base = False` in `begin_kidnap` is **load-bearing**: a stale `True` would fire a phantom base breach the moment the carrier reaches the spawn tile.

> ⚠️ The building is **left standing as a plain dead building** and **revives at payday like any other kill** (user decision). Every payday slot therefore treats it identically: slot 7 explodes a kidnapped booster, slot 8 tears down a kidnapped wall-builder's perimeter, slot 10 restores it, slot 9 rebuilds the building itself.

The carried sprite is one extra `RenderItem` from the `Kidnap` component, offset `(-0.25, +0.25)` tiles: pure iso arithmetic that moves it 16px left at zoom 1 with zero vertical change, leaves `wx+wy` (the depth) identical, and raises `wy` so the carried building sorts **after** the carrier and draws in front of it.

5.9 Combat

`game/enemies/combat.py::resolve_combat(scene, tilemap, dt, buildings_balance, …)`, called each frame **after** `scene.update`.

```
1  defenders   every "combat"-tagged building:
                 keep the sticky target if alive and in Chebyshev range,
                 else acquire the nearest in-range enemy by Euclidean distance;
                 on cooldown, fire.   Reset interval clamped to
                 DefenceBuildings.globals.min_attack_speed.
2  kidnaps     _resolve_kidnaps: after the defender loop, before base arrivals
3  base hits   an enemy with PathAgent.reached_base subtracts dmg and despawns.
                 ⚠️ Exactly ONE base arrival per frame, then bail (prototype-exact).
4  deaths      dead enemies despawn
```

This is the **first writer of `RoundStats`** (`dmg_dealt_this_round` on shooters, `dmg_taken_this_round` on targets).

Three firing paths, dispatched by capability component, never by class:

COMPONENT PRESENT	PATH
`BeamAttacker`	instant hitscan, highest-HP targeting, damage ramp, post-kill re-acquire pause
`SplashAttacker`	arcing `ProjectileAOE` to a fixed ground point via `_predict_lead`, splash on impact, cosmetic crater
neither	plain homing `Projectile`

Projectiles travel then deal GUARANTEED damage on arrival if the target is still alive. A shot in flight is wasted only if its target dies first, never a collision or accuracy miss. Travel time = `distance / projectile_speed_tiles`.

> ⚠️ **The range GATE measures to the footprint's NEAREST TILE; everything else measures from the block CENTRE.** A centre-only gate meant every tile *outside* a 2×2 block sat at Chebyshev ≥ 1.5 from the centre, so a range-1 defender standing adjacent to a boss could never target it, while the boss's own block-and-attack scan hit that same defender fine. Acquisition tiebreak, mortar splash measurement and predictive lead all keep the centre.

> ⚠️ **PERF, load-bearing:** the footprint offset is resolved **once per enemy per frame** (`resolve_combat` builds `targets = [(enemy, off), …]`), never inside the (defender × enemy) pairwise loop. `get_component` is a linear isinstance scan; doing it per pair cost **~9 ms of a 16.7 ms frame** at 50 defenders × 300 enemies. `_chebyshev` also *skips* a zero offset rather than running the block-clamp with a zero span, keeping the N=1 expression in integer arithmetic.

> ⚠️ **Anchors never move flight time.** A defender's optional `muzzle` anchor shifts only *where* the projectile visually spawns; flight time is always computed from the shooter's **unmodified** `transform.world_pos`, passed in as `origin`. Damage arrival timing is provably invariant under any muzzle value.

Tile-condition modifiers are keyed on the tile the enemy last *arrived* at. `PathAgent` refreshes `_current_condition` when `Movement.index` advances, reading `waypoints[index-1]`, gated `index ⩾ 2` because waypoint 0 *is* the spawn tile (whose condition never applies, prototype-exact).

> ⚠️ **The speed floor is not a nicety: the old `max(0, …)` clamp was a LATCH.** The penalty is a flat 0.4 tiles/s and the boss moves at 0.3–0.45, so eras 0–3 computed *exactly* 0.0, and a unit at speed 0 never advances `Movement.index`, which is the only thing that refreshes `_current_condition`, so it stayed 0 forever. The boss was the one unit slower than its own terrain penalty and it **froze solid on the first forest tile.** The formula is now `max(real × min_speed_fraction, real - penalty)`: at the shipped `0.5` the four normal types are byte-identical and only the boss moves.

5.10 Lightning strike

`game/core/lightning.py` (pure; imports `engine.core` only).

- ⚠️ **Boots LOCKED.** `lightning_level` is seeded **0** every run. Placing a Storm Priest is the ONLY way to reach L1. There is **no love-priced upgrade any more**: leveling past L1 is driven entirely by the placed Storm Priest's own tier (tier 1/2/3 → lightning level 1/2/3, latched so a re-sync never lowers it).
- **Damage is a Euclidean circle in the PROJECTED PIXEL plane**, not Chebyshev and not tile-space Euclidean: both points go through `world_to_screen` and the threshold is `radius_tiles * tile_w/2 * zoom` (pan cancels in the delta, zoom scales linearly, so no iso math escapes `engine.coords`).
- The cooldown is spent **unconditionally**: a whiff still pays and still shows VFX. Kills flow through the next `resolve_combat` → normal XP/kill path.
- Cooldown ticks **only** in `pre_sim`'s ENEMY branch, on the host's sim dt (speed-scaled, pause-frozen); never reset by round end or a tier sync.
- Since the Storm Priest dropped the `"combat"` tag it no longer earns its attack pose through combat, so `strike()` is what flashes it: `LightningCaster.trigger()` sets the animator to `"attack"` for 0.4 s. Found via `scene.by_tag("lightning_source")`, duck-typed: no `game.buildings` import.

5.11 Boss

- **Boss rounds** are every `Boss.round_interval`-th round. The boss entry's `tier` argument **is its era** (`round // interval - 1`, clamped).

- ⚠️ **Hunts buildings, hole LAST.** Paths via `find_path_to_nearest_non_base_building` with `repath_on_kill=True`. The fallback to `find_path` when no non-base building is alive is the ONE way `goal_is_base` ever flips true. Before this fix it destroyed 2 of 8 buildings on a scripted board; it now destroys 8 of 8.
- ⚠️ **The target is watched every frame, not just on the blocked→unblocked edge.** It re-paths the moment the target dies, to us *or* to a defender while we were still walking to it, which on a crowded boss round is the common case.
- **Death swarm** is just one instance of the generalised `death_spawn`: 5 per-era rows, `at_hp_fraction: 0.0`, `spawn_hp_fraction: 1.0`.
- **Boss cutscene.** `end_turn` snapshots love every round (for Boss3A) and, on a boss round, lives + an announce marker. `_begin_round_end` queues `{boss_num, outcome}` (outcome = lives vs snapshot). At ROUND_END expiry the cutscene beats a pending level-up. `resolve_boss_cutscene(option, scene)` applies the stack, appends to the per-run `boss_choices` history (no disk persistence), then chains → LEVELUP (if pending) → payday, exactly once.
- **Bonuses** (`boss_bonuses.py`, pure): the prototype's module *without* its global singleton: the six stack counters live in `RunState.boss_stacks`, so a fresh run is the reset. Bonus **magnitudes are code constants**; everything else reads balancing.

5.12 Tutorial

Two halves, and the split is the whole design.

`engine/tutorial.py` (pure, generic). `Step` is a frozen dataclass where **every field is an opaque string**; `TutorialSequencer` offers `current` / `active` / `finished`, `advance(event_id)`, `revert(event_id)`, `skip()`, `allows(action_id)`, `highlight_ids()`, `message_id()`, `flags()`. It knows nothing of tiles, buildings, cards or love. A `"flute"` check inside this module is a layering violation.

`game/tutorial/director.py`. Binds every opaque id to a real thing. This is where "flute"/"economic"/"confirm" vocabulary lives.

> ⚠️ **The gate sits in the UI layer, not the placement seam.** `place_building` and `registry.py` are untouched. Three choke points:

1. **The message branch** in `handle_world_click`: while a message is visible the box consumes every click (highest priority bar GAME_OVER).
2. **`_tutorial_allows_panel_click(mx, my)`** wraps both `panel.handle_click()` call sites. ⚠️ It checks permission **only when the click actually lands on the Confirm button or a construct card**; every other click inside the panel (close, cancel, the name box, the dice reroll) passes through ungated: a deliberate simpler reading over the stricter prose, flagged rather than silently guessed.
3. **`Session.tutorial_gate`**: an optional host-set callable checked in `end_turn()`. The dev-convenience `Space` key goes through the same `Session` check, so it cannot bypass the chain.

> ⚠️ **The tutorial's scripted round is round 0**: real enemy spawning begins at round 1 exactly where it always numerically started, whether the run went through the tutorial or skipped it. The seed is host-side (`build_gameplay`), applied only when the director is active, so a bare `Session` in a logic test still starts at 1. Three sites gained explicit `round_num ≠ 0` guards (boss announce marker, boss cutscene queue, spawner composition).

> ⚠️ **There is no "force a loss" mechanic.** The scripted round has no defence building yet, so the first enemy that reaches the hole IS the scripted loss. The only addition is a **free-loss waiver** gated on a pure read (`charges_life_on_base_hit`) that never mutates the sequencer.

⚠️ **The close-panel hint is a banner, not the modal message box.** `TutorialMessageScreen` swallows every click while visible: using it for a hint that instructs a right-click would block the very right-click it teaches. `submit_tutorial_banner` has no hit-test and consumes no input.

⚠️ **D6 zero-overhead contract:** an inactive/skipped/finished tutorial costs exactly **one `finished` bool check** per gated call site. Both the director and the sequencer fast-path every query to the "always allowed" answer once finished.

⚠️ A missing or invalid tutorial script produces ONE logged warning, an empty already-skipped sequencer, and `active = False`: **never a raise**, so an old or unpainted map is fully playable from frame 1.

5.13 UI

Layering rule: `game.ui → game.core` is ONE-WAY. `game/ui` is **pygame-free** (a source-scan purity test guards it); visuals go out as the engine HUD layer. `shell.py` therefore lives in `ui`, not `core`: the reverse would be circular.

Screen inventory (14). Every screen mirrors the same construct → layout → update → hit → submit template:

SCREEN	ROLE
`main_menu`, `pause`, `settings`, `credits`, `add_name`, `game_over`	the shell's menus
`hud`	love pill, XP bar + level, lives, tile counter, income, phase banner, round, End Turn, Pause
`building_ui`	the right-side panel: **unlock / construct / upgrade / base_info** modes + the `ConstructPreview` modal
`levelup`	the 1-of-3 reward modal
`boss_cutscene`	the A/B story choice modal
`cheat_menu`	`Ctrl+L` debug console
`game_log`	the fading bottom-left message log
`overlays`	the RANGE / HEATMAP toggle pills
`tutorial_message`	the guided-tutorial message box

The click ladder (`main.py`, highest priority first):

```
GAME_OVER
tutorial message box (while visible)
cheat menu (consumes ALL input while open)
LEVELUP / BOSS_CUTSCENE modal
End Turn button
overlay toggle pills
building panel
world (tile pick / lightning strike)
```

⚠️ **HUD submission order is panel → button → text.** The HUD layer has **no depth sort**: first submitted is furthest back. Deliberate exceptions (the hovered terrain tooltip, an active-toggle highlight ring) stay commented at their call site.

Overhead HP bars. Two passes, both hiding the bar at full HP. Widths are per-type class attrs (walker/raider 14×2, siege 24×2, boss 48×4). ⚠️ **The lift is computed, not a constant**: since a sprite's on-screen size derives from its tile footprint rather than its sheet, a lift baked from sheet pixels floats. `_sprite_top` measures the real drawn top edge via the engine's own `fit_factor`. Bars from enemies sharing a tile stack upward 4px per slot.

Anchor resolution: "anchor wins outright." An authored `hp_bar` anchor **replaces** the computed baseline rather than nudging it. ⚠️ The measured root cause: the old baseline was *not* where `Renderer.flush` actually draws the sprite's centre, so an offset composed on top of it still missed by the same 16px gap. Every anchor consumer now resolves through the one shared `engine.render.sprite_anchor_screen`.

Skinnable widgets. `Button` and `submit_panel` take an optional `skin` slot key → one animated nine-sliced `HudSprite` instead of flat rects. ⚠️ **Unskinned output is byte-identical** and golden-pinned. State→animation row map: pressed/flash → `"pressed"`, disabled → `"disabled"`, hover → `"hover"`, else `"idle"`; a missing row falls back to idle, so a partial sheet is fine.

> ⚠️ **Pixel-perfect clickable surface.** Skinned buttons hover and click only over drawn pixels, via a host-injected seam (`widgets.set_skin_hit_test`). The seam queries the **`("idle", 0)` canonical silhouette**: using the hover row would make the cursor oscillate over a hole in it. Unset seam or `skin=None` = rect-only.

Screen skinning. Each screen names its fixed widgets in an `ids` dict `{name: (kind, widget)}`. `layout()` rebuilds `ids` from the **default** geometry and then calls `skinning.apply(screen_id, ids)` **last**, so the override wins. ⚠️ **No override, no mutation**: a screen with an absent or empty override file emits the exact HUD-primitive stream it emitted before skinning existed, and that is the golden parity pin.

> ⚠️ **Every ids target must carry a stored, readable `.rect`.** A widget that computes its position inline at submit time exports `[0,0,0,0]` and renders degenerately at the origin in the editor. For a plain text label the convention is `rect = (x, y, 0, 0)`: an anchor point, W/H nominally zero.

> ⚠️ **Dynamic-count content is not individually overridable.** Levelup's option boxes, the construct cards, the boss-history popup body, and the credits rows inherit the screen's `defaults` section instead. Only stable, always-present widgets get an id.

Fonts, palette and strings are DATA.

> ⚠️ **Every consumer reads the palette via `widgets.C_GOLD` attribute access, never `from .widgets import C_GOLD`.** An early-bound import captures the tuple at import time and a later `configure_palette` rebind can never reach it. This applies to **every reference, not just def-line defaults**: a module-level constant copying a colour is the same trap one level up, which is why `hud.py`'s `_phase_color` is a *function*, not a dict.

> ⚠️ The same trap applies to strings: `widgets.cond_label(name)` and `_phase_label_text(phase)` are **functions**, not dicts of resolved text.

`strings.json` covers what a per-widget `label` override structurally cannot: text that varies by runtime/enum state (a phase banner, a win/loss headline) or is built from a template with live values (`"LIVES {count}"`, `"ROUND {n}"`). Every call site resolves via `T(string_id, **kwargs)`, never a raw f-string.

5.14 VFX

`game/ui/effects.py`'s `FloaterManager` is the game-side FX orchestrator. It owns an `engine.vfx.VfxSystem` and delegates the five stateful families to it, while holding the six stateless parameter bundles directly.

Two dispatch models:

MODEL	USED FOR
Watchers: read trigger state live off the scene each frame	muzzle/slash (an `EnemyCombat.cooldown` reset while blocked = an attack landed), building death bursts, HP bars, beams, craters, lightning, projectiles
Drained ledgers: `RunState` lists the UI empties each frame	income, XP, painter, boost, log, enemy-death splatters, splash impacts, projectile hits, defender fire, boss announces

Neither requires a `game/core → game/ui` import in either direction.

The trigger table. `data/balancing/vfx.json`'s top-level `triggers` object has one row per cosmetic event, each `{sprite_slot, procedural}`:

```
building_placed / _level_up / _tier_up · building_destroyed
enemy_attack_melee / _ranged · enemy_death
splash_impact · projectile_hit · defender_fire
```

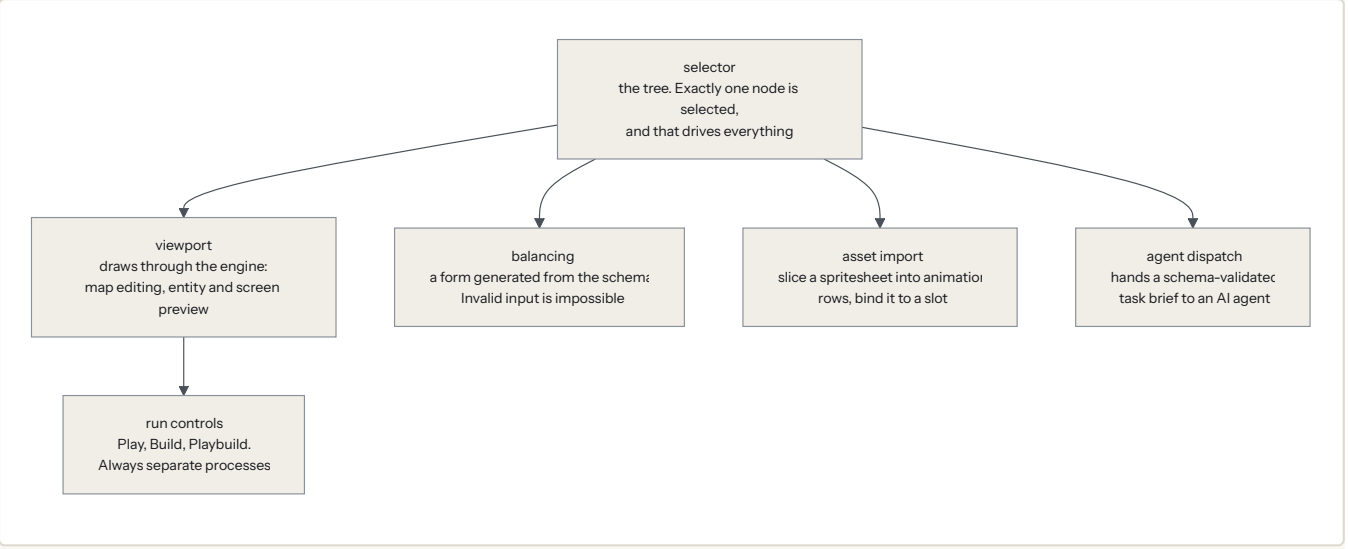
`_play(event, wx, wy, **kw)` is the ONE dispatcher: a bound `sprite_slot` **with imported art** spawns a one-shot `PlayOnceVfx`; otherwise the named `procedural` kind runs; an empty row is a silent no-op. ⚠️ Every shipped row's `procedural` reproduces exactly what that call site did before the table existed: byte-identical on a fresh checkout with no art imported.

Anchoring, visual only. `defender_fire` and both `enemy_attack_*` move to the firing entity's `muzzle`; `building_destroyed` and `projectile_hit` to the target's `impact`. ⚠️ **Two deliberate exclusions:** `enemy_death` (blood) and `splash_impact` (crater) stay **unanchored**: both are *ground decals*, and an `impact` anchor authored at body height would lift them off the ground. `splash_impact` additionally has no owning sprite to read an anchor from.

Part 6: The editor (`editor/`)

Requirements: `SPEC.md` §7 (`ED-*`). Router: `editor/CLAUDE.md`; panel detail in `editor/panels/CLAUDE.md`.

Everything hangs off the selection.



6.1 Shell & the selection model

The editor's core invariant (ED-3): *exactly one selected node at a time* drives the viewport mode, the balancing panel content, and the asset-import context. **New features hang off selection, not parallel state.**

```
┌──────────────────────────────────────────────────────┐
│ Toolbar   Play ▶  Build  Playbuild  Refresh Layouts  │
│           Save Balancing Changes   Version History   │
│           Summon a Drunken Robot   ☐ Dark mode       │
├──────────────┬──────────────────────┬────────────────┤
│  SELECTOR    │       VIEWPORT       │  RIGHT STACK   │
│  (tree)      │  engine/render → pygame │  Details /  │
│              │  Surface → QImage → QPainter │ MapDetails / │
│  categories  │                      │  ScreenDetails │
│   ▸ groups   │  3 modes: entity preview /  │ Palette /  │
│    ▸ leaves  │  tilemap / screen    │  Anchors       │
├──────────────┴──────────────────────┴────────────────┤
│  BALANCING FORM  (auto-generated from the domain's schema)  │
└──────────────────────────────────────────────────────┘
```

Hard rules.

- ⚠️ **One render path.** The viewport draws through `engine/render` into an embedded surface. **QPainter never draws tiles**: it only blits the converted frame. What the editor shows is what the game draws.
- A *second `Renderer` instance* (the VFX preview, the sheet preview) is **not** a second render path. The ban is on a second QPainter-drawn surface of game content, not on a second orchestrator object.
- All `data/` writes go through the schema-validating writer. Invalid input must be **unrepresentable** in the forms.
- ⚠️ **Every new editor module MUST be added to `test_editor_viewport.TestPurity`'s import list**: that is the layering guard.

Composite selection. tree node × Details subcategory dropdown (tier, or the concrete slot for flat groups) × LevelBar index → ONE slot key, resolved by the **pure, Qt-free** `editor/selection.py`.

6.2 Panel inventory

MODULE	ROLE
`panels/selector.py`	the merged tree. Top-level = registry categories in `slots.json` order. Children = registry groups. **Stops** at the deepest group whose children are all leaves (a building *type*): tiers/levels never appear. ● marker = has assigned art. Right-click a category root → "Add new X…" from the form specs.
`panels/viewport.py`	the render surface + all three modes + input
`panels/balancing.py`	the auto-generated form (§6.5). **Home of the `_NoWheel*` widgets.**
`panels/details.py`	asset import + per-slot entry editing
`panels/level_bar.py`	tier/level index strip + "+ Variant" / "+ Type"
`panels/palette.py`	tilemap-mode brushes (4 mode pages: gametiles, decoration, base/markers, tutorial)
`panels/map_details.py`	map lifecycle: New / Duplicate / Save / Set Active / Delete
`panels/screen_details.py`	screen-mode widget property editing
`panels/anchors_panel.py`	manifest anchor handles
`panels/sheet_picker.py` / `sheet_preview.py`	choose and preview an existing sheet
`panels/vfx_preview.py`	live procedural-VFX preview per family
`panels/game_theme.py`	fonts / palette / custom font import
`panels/strings_panel.py`	the UI string table
`panels/cutscenes.py`	cutscene registry + import
`panels/tutorial_panel.py`	tutorial script authoring
`panels/_screen_primitives.py`	the re-implemented unskinned widget look (sanctioned drift)
`panels/_screen_rules.py`	which override controls apply to which widget kind

Editor-wide convention. ⚠️ **Every value widget anywhere in the editor** (spinboxes, combos, in every panel) imports `_NoWheelSpinBox` / `_NoWheelDoubleSpinBox` / `_NoWheelComboBox` from `balancing.py` (their one home). The mousewheel is **navigation-only** everywhere in the editor; it never edits a value.

6.3 The three viewport modes

MODE	ENTERED BY	SESSION	RIGHT PANE
Entity preview	any entity/slot leaf	-	Details
Tilemap	a map leaf under `map ▸ Maps`	`MapSession`	MapDetails (+ Palette)
Screen	a screen leaf under `ui ▸ Screens`	`UIScreenSession`	ScreenDetails

The three are structurally parallel end-to-end: session / viewport mode / right-pane panel / dirty-prompt / undo routing. ⚠️ Window-level Ctrl+Z / Ctrl+Y **route** to the active session's stack.

Entity preview. The entity rendered by the real engine pipeline, idle by default, with an animation dropdown, a range-radius overlay from its balancing values, and a grey X when there is no asset.

Tilemap mode.

- ⚠️ **Painting is pure-model first.** `editor/tilemap_ops.py` (no Qt) mutates the doc and returns `[(col,row,old,new), …]` change lists. The viewport only translates mouse events; **all** cell picking is `screen_to_world` → floor. Strokes Bresenham-interpolate between move events so fast drags do not gap.
- Tools: paint · erase · line · rect · bucket · picker · **none** (default-armed; structurally cannot paint, and a left-drag under it pans).
- Layers with eye toggles: terrain · zone tint · base · deco · start_area · camera_start · tutorial markers.
- Single-object brushes: the hole, camera start, the 2×2 starting area, the two tutorial markers. ⚠️ **Start area and tutorial markers render as `submit_overlay_lines` OUTLINES, never sprites**: the engine emitters deliberately do not emit them.
- **Undo scope:** one command per *stroke* (press→release coalesced, including line/rect/bucket), base move, deco place/remove, marker place/move/erase, display-name edit.
- **Only** MapDetails' "Set Active" writes `data/maps/active_map.json`.

Screen mode. Renders `screen_defaults.json` + the screen's override file. ⚠️ Every `push_*` goes through ONE `_DocFieldCommand` carrying **full old/new values, never a delta**, and `None` means "no override", i.e. the key is **absent**, never JSON `null`. Clearing prunes now-empty parent containers so a fully-reset widget disappears from the doc rather than lingering as `{}`.

> ⚠️ `data/ui/screen_defaults.json` not existing is **not** an error path: it degrades to `{}` and the placeholder handles it.

6.4 Asset import

Full parity with the prototype importer (ED-40), for **every** slot category:

```
pick sheet PNG
  → grid check (off-grid warning, not a refusal)
  → rows = animations, row 0 forced to idle
  → per row: animation name · fps · hidden-frame toggles · loop range × count
  → entry-level offset X/Y
  → optional nine-slice margins, optional anchors, optional tint_overlay
  → live animated preview honouring playback_order
  → Save → copy PNG to data/sprites/imported/<slot>.png
         → write_validated(asset_manifest.json)
  → the entity preview reflects it with no editor restart
```

"Use Spritesheet…" points a slot at another slot's PNG and copies no bytes. **"Clear to placeholder"** removes the entry and, if refcounting says no other entry points at it, unlinks the PNG.

6.5 Balancing form

Auto-generated by recursing the domain's JSON Schema:

SCHEMA NODE	WIDGET
object	`CollapsibleSection` (depth-1 expanded, deeper collapsed)
array of objects	one collapsed sub-section per index, titled `[i] — <name>`
array of scalars	one row per index, **fixed length**
integer / number	`_NoWheelSpinBox` / `_NoWheelDoubleSpinBox`, **range from schema `minimum` / `maximum`**
enum	`_NoWheelComboBox` with typed `itemData`

SCHEMA NODE	WIDGET
boolean	`QCheckBox`
string	`QLineEdit`, commit on `editingFinished`; text shorter than `minLength` restored, not written

⚠️ **`+ Row` / `- Row` on arrays of objects is gated ENTIRELY by the schema**: `can_add = "maxItems" not in node or len < maxItems`, `can_remove = len > minItems`. Every array that predates the feature has `minItems == maxItems`, so it shows no buttons and is byte-identical: that gate *is* the compatibility argument. **Add copies the last row** (the doc validated on load, so a copy is schema-valid by construction); **remove pops the last row**, never a middle one, because these arrays are era-indexed.

⚠️ **Edits are staged, not written immediately.** `_commit(path, value)` mutates an in-memory doc and toggles a pending-change dot by comparing against a baseline deep copy. Signals are connected *after* initial values are set, so populating the form never dirties anything.

Save Balancing Changes is the ONE place that writes: it prompts for a session name + optional description, calls `write_validated`, **and** appends a full-doc snapshot to `data/balancing_history/<domain>.json`. **Version History** lists sessions newest-first; "Load into Editor" replays a snapshot into the live widgets **staged only**: nothing is written until Save is clicked again.

6.6 Theme / strings / cutscenes / tutorial panels

All four follow the same staged-edit + `write_validated` shape as the balancing panel.

- **Theme**: the 7 font presets, the `C_*` palette, custom `.ttf` / `.otf` import, and the active-font pointer. ⚠️ Qt-side font preview uses `addApplicationFontFromData`, not a path, for exactly the same file-lock reason as the game side.
- **Strings**: the flat string table, grouped in the UI by the id's prefix before the first dot. ⚠️ **No editor-side in-process reconfigure**: `game/ui/strings` is off limits to the editor, so the game re-reads at its own next boot.
- **Cutscenes**: the registry + video import + length probing (graceful fallback, never divides by zero, never hangs).
- **Tutorial**: the script's steps, messages and toggles.

6.7 Run controls

CONTROL	BEHAVIOUR
Play ▶	`MainWindow` saves + validates, then `RunControls` launches `py game/main.py` **DETACHED**
Build	PyInstaller via a **tracked** `QProcess`, output streamed to the console pane
Playbuild	launches `dist/HowToBeHuman/HowToBeHuman.exe` **DETACHED**; disabled with a hint when no build exists
Refresh Layouts	re-runs `tools/export_ui_layouts.py` through the same tracked-process infrastructure (so it and Build cannot overlap)

⚠️ **Play and Playbuild are detached, not tracked.** A tracked `QProcess` parented to the editor crashed live with `RuntimeError: Signal source has been deleted` when the long-running GUI process finished. A detached launch has no Qt object to outlive. Only a one-line `launched(which, started_ok)` signal fires: no output streaming.

⚠️ **The detached child gets `_real_window_environment()`**, which strips `SDL_VIDEODRIVER` / `SDL_AUDIODRIVER=dummy`. Without the strip, Play ran the real game (fps printed to console) but rendered into an invisible surface. **Found live, not by automated tests.**

⚠️ **Build needs `PYTHONUNBUFFERED=1`** injected: Python fully block-buffers stdout once it is not a tty, so without it the console shows nothing until exit.

⚠️ **Frozen-exe data path** uses `sys._MEIPASS`, not a `sys.executable`-relative path: PyInstaller 6.x onedir nests `--add-data` under `_internal/`, not beside the exe.

6.8 Agent dispatch: "Summon a Drunken Robot"

The editor is also the launcher for AI agent tasks.

⚠️ **Forms are DATA, not code.** One spec per thing-type in `data/agent_forms/<id>.json`. `AgentFormDialog` renders a spec: title/description → a **built-in free-text box** (never a spec field; every form gets it free) → one row per field → git options → Dispatch. There is exactly **ONE** `field["type"]` → widget switch. **Adding a form means adding a JSON file, never a dialog class.**

Specs load **fresh on every launcher open**, so a spec an agent just wrote appears with no editor restart.

Four dispatch modes, each passing the literal slash command as the agent's opening input so the skill loads directly. Precedence: **admin > handoff > plan > tweak**.

MODE	COMMAND
form	`/dispatch <handoff-path>`
plan	`/setcurrentplan <name>` or `/createplan <brief>`
small tweak	`/smalltweak <task>`
admin	blank `claude` (no input, no scope)

Handoffs are written through `write_validated` against `dispatch_handoff.schema.json` into **gitignored `.claude/dispatch/`** (transient agent I/O, not `data/` content). The launcher prunes completed handoffs on every open.

⚠️ **The editor never writes root `PLAN.md` or anything under `planning/`.** It *reads* the active plan and offers a picker; the spawned skills do the writing.

⚠️ **Import direction:** `agent_form_dialog` imports `spawnclaude` at module top; `spawnclaude` imports `AgentFormDialog` **lazily inside `_open_form`**: top-level both ways is a cycle.

Part 7: Cross-cutting invariants

The consolidated "do not break this" list. Each has already been stated in context; this is the index.

7.1 Layering

1. `game/` ↮ `editor/`: never, in either direction.
2. All iso math in `engine/coords`.
3. All `data/` writes through `write_validated`.
4. `engine/` contains no game vocabulary.
5. `game/core` ↮ `game/enemies`; cross-boundary via optional callbacks and opaque payloads.
6. `game/map` never imports `game/buildings`; it duck-types occupants.
7. `game/ui → game/core` is one-way.
8. `game/ui` is pygame-free; `game/core`, `game/map`, `game/buildings`, `game/enemies` too.
9. Every new editor module joins `TestPurity`'s import list.

7.2 Performance

A regression in any of these drops a 1024^2 map to ~2 fps.

1. **Every tile-state write goes through `TileMap.set_tile_state`** (keeps the `_by_state` index consistent; HUD tile queries are O(result), not full scans). Four full-map scans per frame on a 1024^2 map measured **~630 ms/frame**.
2. **Placement occupancy is incremental**; `sync_occupancy` is rebuild-only.
3. **No full-map scans on routine actions**: `_find_2x2` uses an expanding-window search, byte-identical to the old full scan.
4. **Ground terrain draws through the scrolling `GroundCache`**, fed the `band_render_items` **diagonal-strip** emitter (not `visible_render_items`), and invalidated once per zone change, never per frame.
5. **One Dijkstra per topology change per footprint**: every weight/blocking mutation must bump `_path_version`.
6. **`depth_key` must keep the layer as its primary key**, or the ground cache breaks.
7. **Footprint offsets resolve once per enemy per frame**, never inside the pairwise combat loop.
8. **`footprint == 1` keeps its inline fast branch** in the pathfinder.
9. **Condition art is one RenderItem per visible tile with art, every frame.** Measure before shipping grass art: `GroundCache` cannot absorb it.
10. GC is frozen after `build_gameplay`, gated to windowed runs.

7.3 Determinism & RNG

- **Seed the RNG in any test whose outcome depends on it.** The spawner takes an injectable `rng` precisely so tests are deterministic; a test using the bare `random` module failed roughly one run in ten for reasons unconnected to what it measured.
- Every `engine/vfx` emitter takes an **injected** rng.
- `TileMap(doc, balance, rng=…)`: module `random` live, `random.Random(seed)` in tests, **`None` skips the roll entirely** (the all-GRASS fixture mode).

- `SpatialGrid` returns in **insertion order**.
- Composition-order changes (e.g. adding a group before `_formation_group`) shift the rng draw sequence and break wave fixtures.
- `data_io` dumps deterministically; `export_ui_layouts.py` is deterministic, so merge conflicts resolve by re-running it.

7.4 Fail loud vs degrade gracefully

The split is a rule, not a habit:

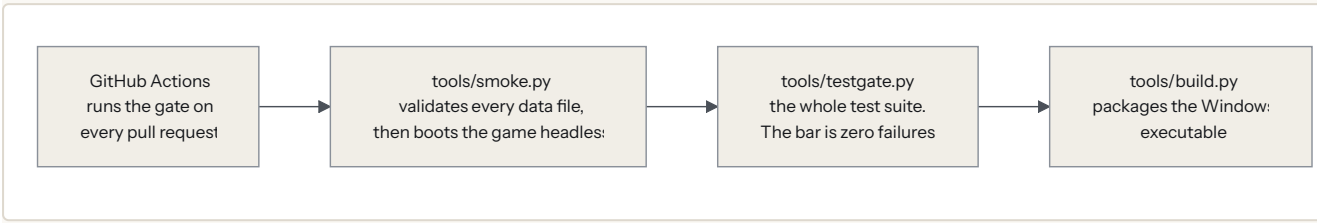
CATEGORY	POLICY	EXAMPLES
Structural / config data	**FAIL LOUD (D-2)**	map files, `active_map.json`, `slots.json`, `geometry.json`, all six balancing domains, `fonts.json`, `palette.json`, `strings.json`, `active_font.json`
Art	**DEGRADE (E-37)**	missing manifest → empty; corrupt entry → warn + skip; missing slot → grey-X placeholder; **rendering never raises on missing art**
Optional subsystems	**SKIP SILENTLY**	audio (swallows all exceptions), cv2/cutscenes (`enabled=False`, `done=True`)
Scripted content	**ONE WARNING, then inert**	a missing/invalid tutorial script → empty already-skipped sequencer, `active=False`
Editor reads	degrade	`screen_defaults.json` absent → `{}`; a form-spec load failure → no context menu + one stderr line (an exception inside a Qt event handler can abort the process: a right-click must never kill the editor)

7.5 State ownership: who owns what

STATE	OWNER
Every gameplay **value**	`data/**.json`
Run state (love, lives, round, phase, ledgers, boss stacks)	`RunState`
Per-object gameplay state	**components**, never subclass attributes
Tile zone / occupant / condition / walls	`TileMap`
Camera pan + zoom	`CoordinateSystem`
Combat-speed selector + round gates	`Session`
Where combat speed **applies**	the host (`main.py`)
The selected node	the editor's `MainWindow`
Undo	one `QUndoStack` per session, routed by mode
Cosmetic fade clocks (crater, lightning, corpse, play-once)	scene components, **not** `VfxSystem`

Part 8: Tooling, tests, build

The gate.



8.1 The exit gate

```
py tools/smoke.py           # data validation + 5-frame headless boot
py tools/testgate.py check  # the suite. Read the ONE line it prints.
```

> ⚠️ **The gate is ZERO.** `GATE PASS` or you are not done. There is no baseline to measure against and no "pre-existing failure" to tolerate: if a test is red, you broke it. (It was not always so: the suite once carried 18 permanent failures and the gate was a *diff* against a number that lived in prose and had drifted three ways.)

RULE	WHY
Never re-run the suite to discover what was already broken	that waste is exactly what the gate deletes
`--affected` runs the Graphify blast radius ∪ the `core` tier while iterating	the full check runs **once**, at the end
An unexpected skip is a failure	a test that quietly stops running is indistinguishable from one that passes
So is a failing subtest	the gate reads pytest's `SUBFAILED` lines, which it once ignored while printing PASS
A red test clearly outside your diff's blast radius: **note it and stop**	surface it, do not dig
Never paste raw gate output into a report	collapsing it to one line is the whole point

Tiers (declared in `conftest.py`'s `TIERS` table): `core` (fast, ~800 tests) · `editor` (Qt, slow) · `meta` (agent scaffolding). CI runs the whole suite: there is no excluded tier. ⚠️ **Every new test module needs a tier**; `test_tiers.py` fails if you forget, because an unmarked module would silently never run under a marker-selected gate.

8.2 Test discipline

Two rules, both learned the hard way:

1. ⚠️ **Every widget constructed in a test must be DESTROYED.** Subclass `QtCase` and wrap it: `self.track(MainWindow(...))`. `addCleanup(w.close)` is **not** cleanup: Qt's `close()` *hides* a window, it does not destroy it. Each leaked `MainWindow` kept ~2,972 widgets alive and made the next one slower to build, which is how the suite became quadratic: **17m 15s for what now takes 2m 31s.** Leaked widgets also outlived their tests and **wrote into the repo's `data/`.**

2. ⚠️ **Never assert against live `data/` content, pin the fixture.** Use `TempDataCase.unassign_slot` / `unassign_family` / `drop_slot_variants` and `MapModeCase.set_active_map` to *guarantee* the state you need.

8.3 The tools

TOOL	ROLE
`tools/smoke.py`	SDL dummy → validate every `data/` file against its schema → construct the game → 5 frames → OK **(T-2)**
`tools/testgate.py`	the one-line gate; `check --affected` for the blast radius
`tools/build.py`	PyInstaller `--onedir`; calls `validate_data()` first; invoked as `[sys.executable, "-m", "PyInstaller", …]` **(T-4)**
`tools/export_ui_layouts.py`	regenerates `data/ui/screen_defaults.json`
`tools/bake_ui_sheets.py`	bakes the default UI button/panel/icon sheets
`tools/data_guard.py`	the `data/` hash fixture
`tools/doctor.py`	environment diagnostics
`tools/render_demo.py`	renders the grey-X grid offscreen for a quick visual check

8.4 CI and branching

- `.github/workflows/tests.yml` gates every PR into `Development`.
- One branch per phase/feature off `Development`; land via PR.
- ⚠️ **Never run destructive git on uncommitted work**: no `git reset --hard`, no `git clean`, no `git checkout -- <file>`, no force-push.
- Never commit `build/`, `dist/`, or any `*.exe`.
- Every PR states a concrete **in-game Quick Test scenario**, not just a checklist.

Part 9: Extension recipes

Each of these has a **skill** that encodes the full pattern (files to touch, order, verify gate). Invoke the skill rather than editing by hand: it is the source of truth and keeps changes consistent. Most are also **forms** in the editor (`data/agent_forms/*.json` is the roster).

TASK	SKILL	TOUCHES
Add a building type	`/add-building`	leaf class + `research.py` row + `registry.py` + `buildings.json` subtree + schema + `slots.json` group
Add an enemy type	`/add-enemy`	thin `Enemy` subclass (**with `_resolve_stats` overridden**) + spawner branch + `enemies.json` block (incl. `footprint`, `registry_group`, `kidnapping`, `death_spawn`) + registry group
Add / change a balancing tunable	`/add-balancing-value`	`balancing/<domain>.json` + `schemas/<domain>.schema.json` (schema **first**), through the validating writer
Add an engine Component	`/add-engine-component`	declared JSON-safe fields + the `on_added` seam + auto-registration + module purity
Add an editor feature/panel	`/add-editor-feature`	hang off the selection model, one render path, writes via `write_validated`, **add to `TestPurity`**
Wire a new renderable category	`/add-asset-importer`	registry category + slots + selector + import path
Add a slot-registry category / balancing domain	`/add-category`	`slots.json` + every place that hardcodes the domain list (there should be none, it is derived)
Replace a sprite for an existing thing	`/replace-visual`	the manifest-v2 asset pipeline
Add an "Add new X" form	`/add-form-spec`	one JSON file in `data/agent_forms/`
Scaffold a command/skill	`/add-skill`	`.claude/commands/`

Manual recipe sketch: a new building type, to show the shape:

1. `data/balancing/buildings.json` + its schema: a new group under the right family, with `tiers[]`, `starts_unlocked`, and `tiers[0].unlock_min_round`.
2. `data/slots.json`: a new `buildings` group with per-tier/level slot keys.
3. `game/buildings/<name>.py`, a ≤10-line leaf: `SUBTREE`, `BUILDING_TYPE`, `TIER_SPRITES`, plus any extra capability component.
4. `game/buildings/research.py`: one `LEAF_CLASSES` entry + one `RESEARCH` row. **Never reopen the level-up roll.**
5. Firing behaviour (if combat) goes in `game/enemies/combat.py`, selected by a marker component: **not** on the building.
6. Import art through the editor.
7. `py tools/smoke.py && py tools/testgate.py check`.

Part 10: Known divergences & open questions

10.1 Deliberate divergences from the prototype

AREA	DIVERGENCE
The hole	**Lives**, not HP.
Raiders / siege	Seek the **base**, not their prototype prey. They still eat what stands in their lane; they just do not hunt. The re-path machinery exists (shipped for the boss): arming it is a one-line change if ever ruled desired.
Cheat menu	`Ctrl+L`, not `Ctrl+P` (bare `P` is quick-skip here).
`SETTINGS_PAUSED`	Does not exist. One `SETTINGS` state reused via `settings_caller`.
Enemy sprite spread	Not ported. HP-bar slots therefore go out compactly instead of leaving gaps.
XP mascot face	Never ported. (The `xp_icon` gap itself is closed.)
Lightning FX	Not force-cleared at round end; they age out like craters, so a strike in the final combat instant lingers ≤0.4 s into REBUILDING.
`_try_enter_wall_attack`	The prototype's degenerate nearest-wall fallback is intentionally not ported: no practical gameplay difference.
Balancing values	**Free.** The parity gate is deleted. A number that differs from the prototype is a design decision.

10.2 Known cosmetic gaps

- **Even-footprint sprite offset is FIXED** for the render layer (ER-5's `block_center_offset`), but any *new* consumer that places something over a sprite must call `block_center_offset` too, or it will float by 16px per extra tile.
- Enemy low-HP blood blotches are approximated by the engine tint path, not per-pixel sprite mutation.
- Splatters and craters draw in the overlay pass, i.e. **over** sprites; the prototype drew them under buildings.
- Particle velocities are eyeballed around the prototype's presets (life, count and colours are exact).
- Overlay diamond **borders** are opaque: `OverlayLines` carries no alpha. Fills are exact.
- The settings audio slider is inert (no audio system beyond one music track).
- Only one `pygame.mixer.music` channel exists: starting a cutscene's companion track replaces the background music and nothing restores it.

10.3 Known limitations

- ⚠️ **A death on the wave's LAST frame** can still drop children into a just-ended round in edge cases. The wave-clear check now consults `queued_by_tag`, which closes the common case; the deeper fix is to teach the check about *all* pending spawns.
- `find_path_to_nearest_economic` / `_defence` exist and are queried by nothing.
- `floater_origin` / `status_icon` / `beam_endpoint` anchors are declared and parse-ready but have no read-site.

10.4 Documentation drift found while writing this document

- `game/core/phases.py`'s module docstring says `BOSS_CUTSCENE` is "still reserved, NEVER entered yet". **It is entered** (`session.py:409`, `_begin_boss_cutscene`). The docstring is stale; the enum itself is correct.
- `game/CLAUDE.md`'s TU-6/TU-7/TU-8 sections still say "round 1"/"round 2" in places describing the flute/stone chains. TU-9 moved the tutorial to **round 0**. The sections flag this themselves, but a reader skimming will trip on it.
- `README.md` ends with "Nothing is runnable yet, see PLAN.md phase status." Both the game and the editor run. That line predates the migration completing.
- `MIGRATION_AGENT_READ_FIRST.md` calls Pond impassable. Pond is **expensive** (+9 weight), not impassable: `game/map/CLAUDE.md` records the ruling.

10.5 Open questions

1. Access-limitation ideas (ED-63): pending designer input; must not require restructuring.
2. Whether `ui` / `vfx` warrant further balancing-domain splits.
3. Whether raider/siege prey-hunting should be armed now that the machinery exists.
4. Whether `REGISTRY_GROUP` (Python) and `registry_group` (data) should collapse into one source. They are currently pinned equal by a drift test.

Appendix A: Requirement ID index

`SPEC.md` numbers every requirement. Quick orientation:

PREFIX	SCOPE	COVERED IN
`E-1 … E-5`	coordinates & camera	§3.1
`E-10 … E-15`	game object model	§3.2
`E-20 … E-26`	render pipeline	§3.5
`E-30 … E-32`	physics	§3.3
`E-33 … E-38`	asset system (E-38 **retired**)	§3.4
`D-1 … D-4`	data principles	§4.1
`D-10 … D-12`	balancing (D-11 **removed**)	§4.4
`D-20 … D-22`	maps	§4.7
`D-30 … D-32`	manifest & slots	§4.5, §4.6
`G-1 … G-8`	the game	Part 5
`ED-1 … ED-3`	editor shell	§6.1
`ED-10 … ED-11`	selector	§6.2
`ED-20 … ED-24`	viewport	§6.3
`ED-30 … ED-32`	balancing panel (ED-32 **removed**)	§6.5
`ED-40 … ED-42`	asset import	§6.4
`ED-50 … ED-52`	run controls	§6.7
`ED-60 … ED-63`	agent dispatch	§6.8
`T-1 … T-5`	tooling (T-1 **removed**)	Part 8

Phase tags you will see in code comments and docs (`9A – 9H`, `10A – 10L`, `A1 – A8`, `B1 – B4`, `ER-1 – ER-5`, `ESV-1 – ESV-6`, `TU-1 – TU-9`, `UH-*`, `BP-*`) refer to plan documents in `planning/` (and `planning/completed plans/`).

Appendix B: File map

```
HowtobeHumanFullProd/
├── CLAUDE.md              router for agents: read this first
├── SPEC.md                numbered requirements
├── PLAN.md                generated mirror of the active plan (never hand-edit)
├── README.md
├── conftest.py            pytest tiers + the data/ hash fixture
│
├── engine/                the pseudo-engine  (~4.4k lines)
│   ├── coords/            system, camera, geometry      THE coord authority
│   ├── core/              gameobject, component, transform, scene,
│   │                      health, movement, range_sensor, sprite_animator
│   ├── physics/           grid, occupancy, movement
│   ├── assets/            types, manifest, registry, store, placeholder, nine_slice
│   ├── render/            renderer, backend, item, hud, fonts, ground_cache
│   ├── vfx/               params, particle, emitters, system, play_once
│   ├── tilemap.py         THE map file authority + 3 emitters
│   ├── data_io.py         write_validated / load_validated
│   ├── tutorial.py        generic step sequencer (opaque ids)
│   ├── audio.py  video.py  video_playback.py
│   └── CLAUDE.md          + one per subfolder
│
├── game/                  How To Be Human  (~14.6k lines)
│   ├── main.py            THE entry point: window, loop, input, _World
│   ├── anchors.py         anchor_world_point / projectile_point
│   ├── core/              phases, game_state, session, payday, xp, levelup,
│   │                      lightning, boss_bonuses, balance, names
│   ├── map/               tiles, tile_map, pathfinder, picking,
│   │                      conditions, spawn_deco
│   ├── buildings/         building, registry, research, components, coverage,
│   │                      base_building, defence/defender, economy/musician,
│   │                      meditator, painter, aoe_defence, sun_scorcher,
│   │                      storm_priest, boost, structure
│   ├── enemies/           enemy, spawner, combat, components, corpse, kidnap
│   ├── tutorial/          director.py
│   ├── ui/                shell, hud, building_ui, widgets, effects, overlays,
│   │                      levelup, boss_cutscene, cheat_menu, game_log,
│   │                      game_over, main_menu, pause, settings, credits,
│   │                      add_name, tutorial_message, cutscene_player,
│   │                      skinning, strings
│   ├── PERF.md            the large-map performance playbook
│   └── CLAUDE.md          + one per domain folder
│
├── editor/                the PySide6 editor  (~13.4k lines)
│   ├── main.py            Qt shell, selection model, mode routing, undo routing
│   ├── panels/            selector, viewport, balancing, details, palette,
│   │                      map_details, screen_details, anchors_panel,
│   │                      level_bar, sheet_picker, sheet_preview, vfx_preview,
│   │                      game_theme, strings_panel, cutscenes, tutorial_panel,
│   │                      _screen_primitives, _screen_rules
│   ├── map_session.py  ui_screen_session.py        the two doc sessions
│   ├── selection.py  tilemap_ops.py  registry_ops.py  asset_import.py
│   ├── agent_forms.py  agent_form_dialog.py  spawnclaude.py  plans.py
│   ├── run_controls.py  theme.py  theme_ops.py  domains.py
│   └── CLAUDE.md  panels/CLAUDE.md
│
├── data/                  THE value store
│   ├── schemas/           22 JSON Schemas: the gate on everything
│   ├── balancing/         buildings, enemies, map, ui, core, vfx
│   ├── balancing_history/ per-domain save snapshots
│   ├── maps/              8 maps + active_map.json
│   ├── sprites/           asset_manifest.json + imported/*.png (COMMITTED)
│   ├── ui/                screens/*.json, screen_defaults, fonts, palette,
│   │                      strings, active_font
```

```
│   ├── fonts/              font_manifest.json + imported/*.ttf
│   ├── agent_forms/        the "Add new X" form specs
│   ├── tutorial/  video/   slots.json  geometry.json  display.json
│   └── CLAUDE.md
│
├── tools/                  smoke, testgate, build, exporters, doctor,
│   └── tests/              103 test modules + fixtures
│
├── docs/
│   ├── TDD.md                      ← this document
│   ├── SYSTEM_DIAGRAM_BRIEF.md     ← the visual spec for the system diagram
│   ├── prompt-templates.md
│   └── briefs/                     per-phase implementation briefs
│
├── planning/               plan docs (+ completed plans/)
└── .claude/                agents/, commands/ (skills), dispatch/ (gitignored)
```

Appendix C: Doc map

Read exactly one package doc for your task. They are routers, and the subsystem docs auto-load when you edit inside their folder.

DOC	OWNS
`CLAUDE.md` (root)	task classification, the C2 agent workflow, the exit gate, branching
`SPEC.md`	numbered requirements
`PLAN.md`	the active plan (a **generated mirror**: never hand-edit)
`engine/CLAUDE.md`	cross-cutting engine rules + the top-level modules
`engine/coords/CLAUDE.md`	the coordinate authority
`engine/core/CLAUDE.md`	GameObject/Component/Scene/serialisation
`engine/render/CLAUDE.md`	the render pipeline, anchors, sizing, ground cache, fonts
`engine/physics/CLAUDE.md`	grid, occupancy, waypoint advance
`engine/assets/CLAUDE.md`	slots, manifest, playback order, placeholder, hit-mask
`data/CLAUDE.md`	every format and schema
`game/CLAUDE.md`	the host, cross-cutting rules, perf invariants, the tutorial
`game/core/CLAUDE.md`	round machine, payday, XP, level-up, lightning, boss
`game/map/CLAUDE.md`	tiles, zones, unlocking, conditions, pathfinding, walls
`game/buildings/CLAUDE.md`	the hierarchy, components, research/gating
`game/enemies/CLAUDE.md`	walker, spawner, combat sweep, death spawn, kidnap
`game/ui/CLAUDE.md`	HUD, panel, screens, skinning, strings, FX
`game/PERF.md`	the large-map performance playbook (the "why" + measured numbers)
`editor/CLAUDE.md`	shell, run controls, dispatch, theme, testing rules
`editor/panels/CLAUDE.md`	every panel in detail
`docs/prompt-templates.md`	copy-paste task openers
`docs/briefs/*.md`	per-phase implementation briefs

End of document.